

malloc & vmalloc in Linux

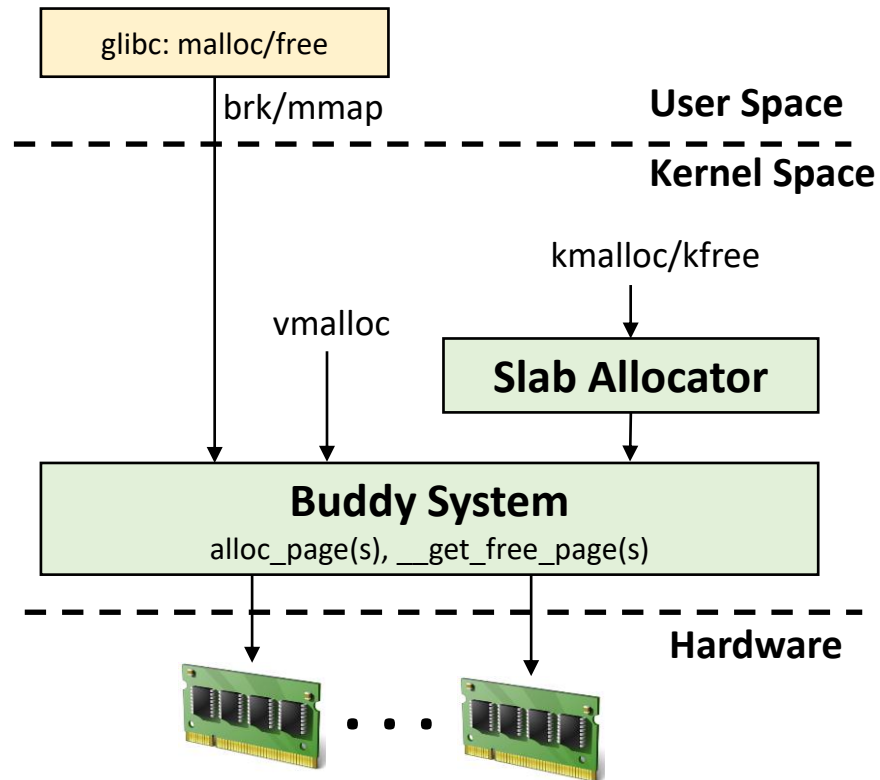
Adrian Huang | Dec, 2022

- * Based on kernel 5.11 (x86_64) – QEMU
- * 2-socket CPUs (4 cores/socket)
- * 16GB memory
- * Kernel parameter: nokaslr norandmaps
- * KASAN: disabled
- * Userspace: ASLR is disabled
- * Legacy BIOS

Agenda

- Memory Allocation in Linux
- malloc -> brk() implementation in Linux Kernel
 - Will *NOT* focus on glibc malloc implementation: You can read this link: [malloc internal](#)
- vmalloc: Non-contiguous memory allocation
- [Note] kmalloc has been discussed here: Slide #88 of [Slab Allocator in Linux Kernel](#)

Memory Allocation in Linux



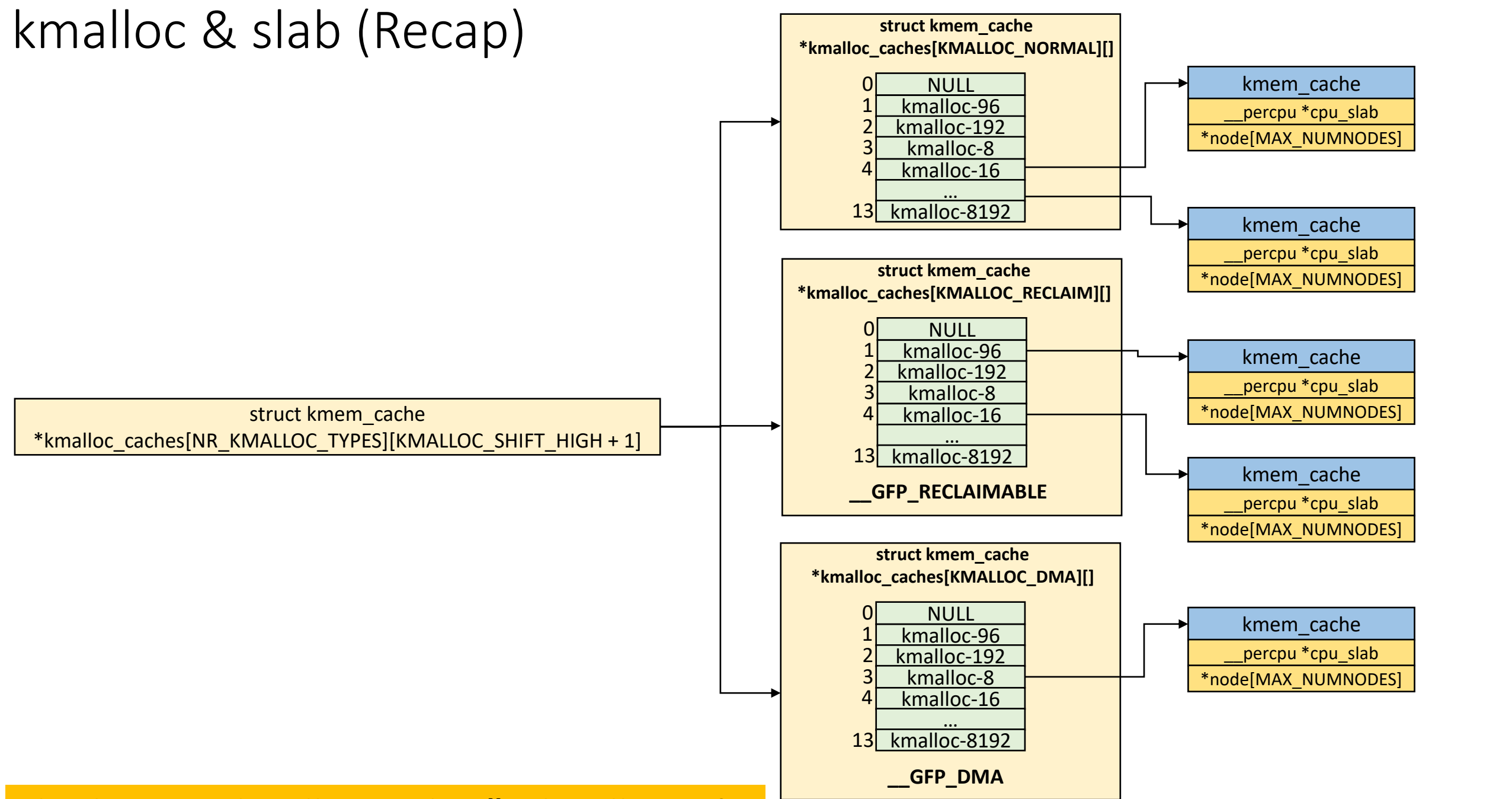
[glibc] malloc

- Balance between **brk()** and **mmap()**
- Use **brk()** if request size < **DEFAULT_MMAP_THRESHOLD_MIN** (128 KB)
 - The heap can be trimmed only if memory is freed at the top end.
 - **sbrk()** is implemented as a library function that uses the **brk()** system call.
 - When the heap is used up, allocate memory chunk > 128KB via **brk()**.
 - Save overhead for frequent system call 'brk()'
- Use **mmap()** if request size >= **DEFAULT_MMAP_THRESHOLD_MIN** (128 KB)
 - The allocated memory blocks can be independently released back to the system.
 - Deallocated space is not placed on the free list for reuse by later allocations.
 - Memory may be wasted because **mmap** allocations must be page-aligned; and the kernel must perform the expensive task of zeroing out memory allocated.
 - Note: glibc uses the dynamic **mmap** threshold
 - Detail: ``man mallopt``

kmalloc & vmalloc

- **kmalloc**: Contiguous memory allocation
- **vmalloc**: Non-contiguous memory allocation
 - Scenario: memory allocation size > **PAGE_SIZE** (4KB)
 - Allocate virtually contiguous memory
 - Physical memory might NOT be contiguous

kmalloc & slab (Recap)



Check `create_kmalloc_caches()` & `kmalloc_info`

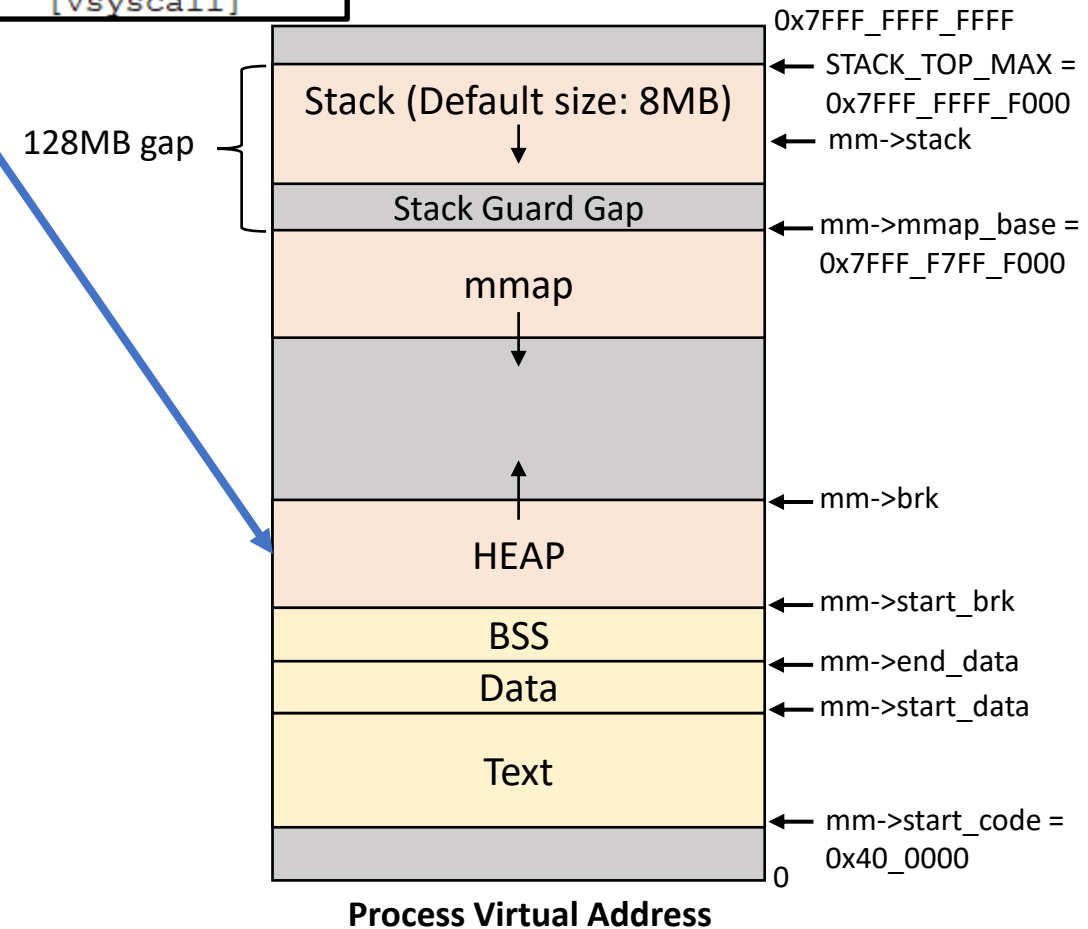
malloc() -> brk() implementation in Linux Kernel

- Quick view: Process Address Space – Heap
- sys_brk – Call path
- [From scratch] Launch a program: load_elf_binary() in Linux kernel
 - VMA change observation
 - Heap (brk or program break) configuration
- [Program Launch] strace observation: heap – brk()
- strace observation: allocate space via malloc()
 - If the heap space is used up, how about allocation size when calling malloc()->brk?
- glibc: malloc implementation for memory request size

Quick view: Process Address Space - Heap

```
/ # cat /proc/44/maps
00400000-00401000 r--p 00000000 00:02 1540 /free_and_sbrk
00401000-00496000 r-xp 00001000 00:02 1540 /free_and_sbrk
00496000-004be000 r--p 00096000 00:02 1540 /free_and_sbrk
004be000-004c1000 r--p 000bd000 00:02 1540 /free_and_sbrk
004c1000-004c4000 rw-p 000c0000 00:02 1540 /free_and_sbrk
004c4000-004e8000 rw-p 00000000 00:00 0 [heap]
7ffff7ffa000-7ffff7ffe000 r--p 00000000 00:00 0 [vvar]
7ffff7ffe000-7ffff7fff000 r-xp 00000000 00:00 0 [vdso]
7ffff785d000-7ffffffffff000 rw-p 00000000 00:00 0 [stack]
ffffffff600000-ffffffff601000 --xp 00000000 00:00 0 [vsyscall]
```

```
(gdb) p ((struct task_struct *) 0xffff888240f70000)->comm
$4 = "free_and_sbrk\000\000"
(gdb) p ((struct task_struct *) 0xffff888240f70000)->pid
$5 = 44
(gdb) p /x ((struct task_struct *) 0xffff888240f70000)->mm->start_brk
$6 = 0x4c5000
(gdb) p /x ((struct task_struct *) 0xffff888240f70000)->mm->brk
$7 = 0x4e8000
```

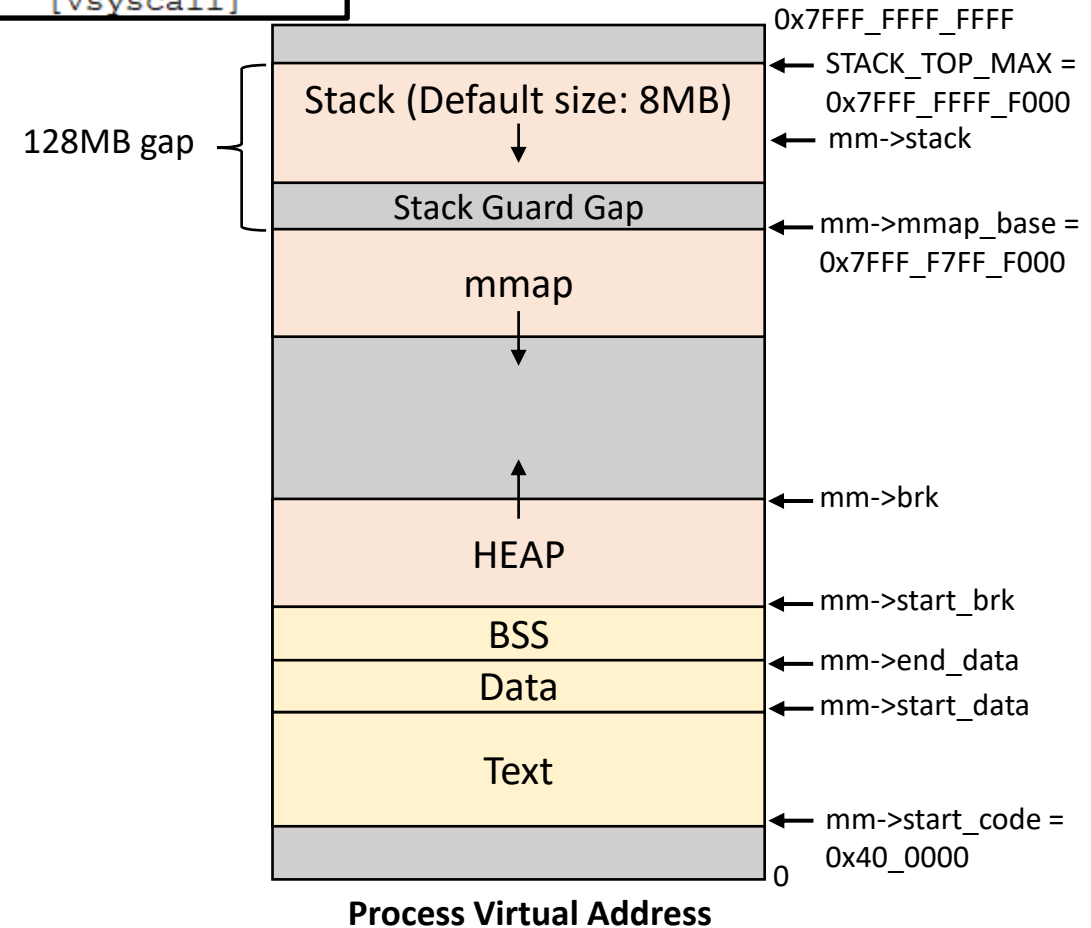


Quick view: Process Address Space - Heap

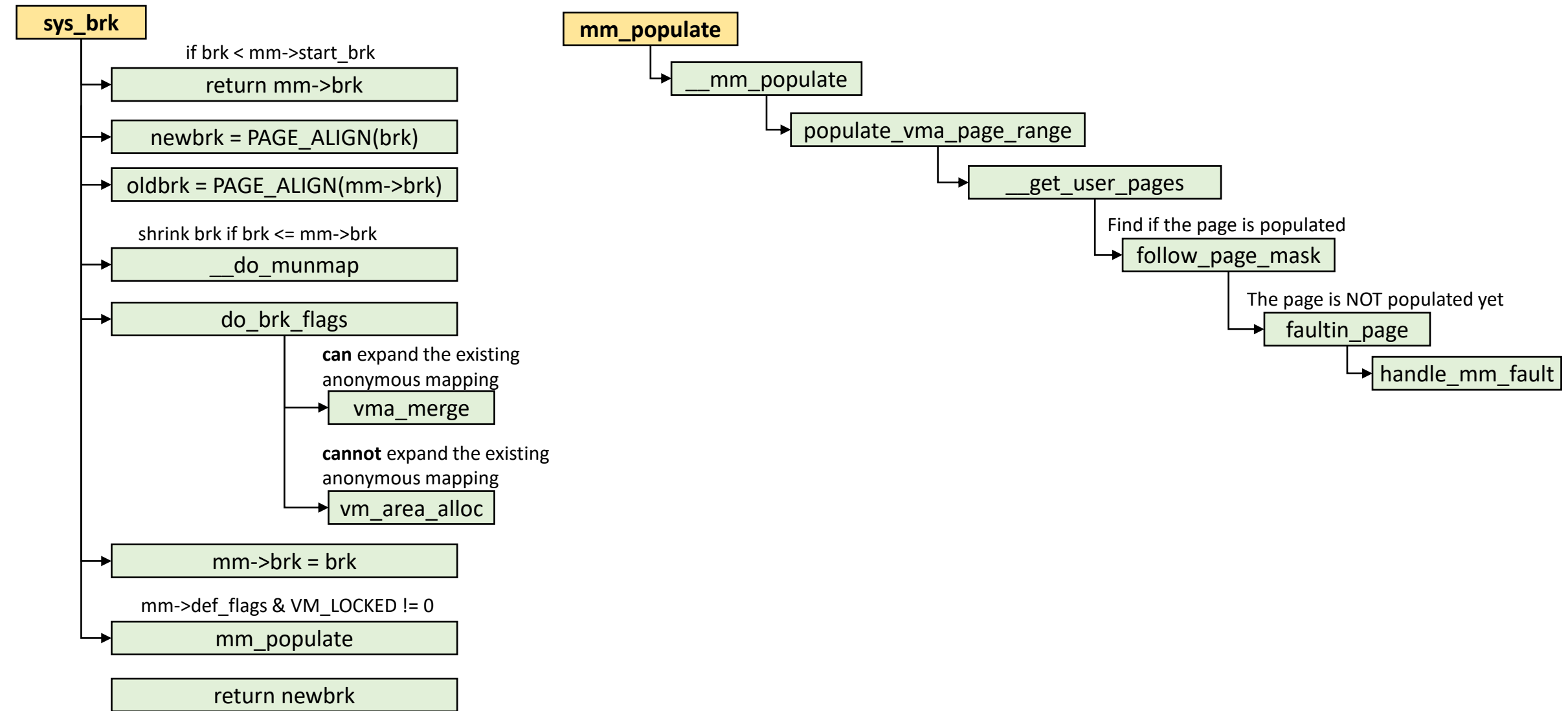
```
/ # cat /proc/44/maps
00400000-00401000 r--p 00000000 00:02 1540      /free_and_sbrk
00401000-00496000 r-xp 00001000 00:02 1540      /free_and_sbrk
00496000-004be000 r--p 00096000 00:02 1540      /free_and_sbrk
004be000-004c1000 r--p 000bd000 00:02 1540      /free_and_sbrk
004c1000-004c4000 rw-p 000c0000 00:02 1540      /free_and_sbrk
004c4000-004e8000 rw-p 00000000 00:00 0        [heap]
7ffff7ffa000-7ffff7ffe000 r--p 00000000 00:00 0        [vvar]
7ffff7ffe000-7ffff7fff000 r-xp 00000000 00:00 0        [vdso]
7ffffff85d000-7fffffff0000000000000000 rw-p 00000000 00:00 0        [stack]
ffffffff600000-ffffffff601000 --xp 00000000 00:00 0        [vsyscall]
```

```
(gdb) p ((struct task_struct *) 0xffff888240f70000)->comm
$4 = "free_and_sbrk\000\000"
(gdb) p ((struct task_struct *) 0xffff888240f70000)->pid
$5 = 44
(gdb) p /x ((struct task_struct *) 0xffff888240f70000)->mm->start_brk
$6 = 0x4c5000
(gdb) p /x ((struct task_struct *) 0xffff888240f70000)->mm->brk
$7 = 0x4e8000
```

Why are they different?



sys_brk – Call path



[By default] Heap (or brk) space is on-demand page

[From scratch] Launch a program: load_elf_binary() in Linux kernel

```
$ readelf -l free_and_sbrk

Elf file type is EXEC (Executable file)
Entry point 0x401be0
There are 10 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
     FileSiz      MemSiz          Flags  Align
LOAD             0x0000000000000000 0x0000000000400000 0x0000000000400000
                0x0000000000000518 0x0000000000000518 R       0x1000
LOAD             0x0000000000001000 0x0000000000401000 0x0000000000401000
                0x00000000000094ac1 0x00000000000094ac1 R E     0x1000
LOAD             0x0000000000009600 0x0000000000496000 0x0000000000496000
                0x0000000000002702f 0x0000000000002702f R       0x1000
LOAD             0x000000000000bd0c0 0x00000000004be0c0 0x00000000004be0c0
                0x0000000000005650 0x0000000000006e60 RW      0x1000
NOTE            0x0000000000000270 0x0000000000400270 0x0000000000400270
                0x0000000000000020 0x0000000000000020 R       0x8
NOTE            0x0000000000000290 0x0000000000400290 0x0000000000400290
                0x0000000000000044 0x0000000000000044 R       0x4
TLS             0x000000000000bd0c0 0x00000000004be0c0 0x00000000004be0c0
                0x0000000000000020 0x0000000000000060 R       0x8
GNU_PROPERTY    0x0000000000000270 0x0000000000400270 0x0000000000400270
                0x0000000000000020 0x0000000000000020 R       0x8
GNU_STACK       0x0000000000000000 0x0000000000000000 0x0000000000000000
                0x0000000000000000 0x0000000000000000 RW      0x10
GNU_RELRO       0x000000000000bd0c0 0x00000000004be0c0 0x00000000004be0c0
                0x0000000000002f40 0x0000000000002f40 R       0x1
```

./free_and_sbrk 1 1

Kernel
load_elf_binary()

GAP	vma: R	vma: R, E	vma: R	vma: R, W	GAP	vma (vvar)	vma (vdso)	GAP	vma (stack)
	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000		vm_start = 0x7fff7ffa000	vm_start = 0x7fff7ffe000		vm_start = 0x7ffff85d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c4000		vm_end = 0x7fff7ffe000	vm_end = 0x7fff7fff000		vm_end = 0x7fffffff000

After launching a program: Question

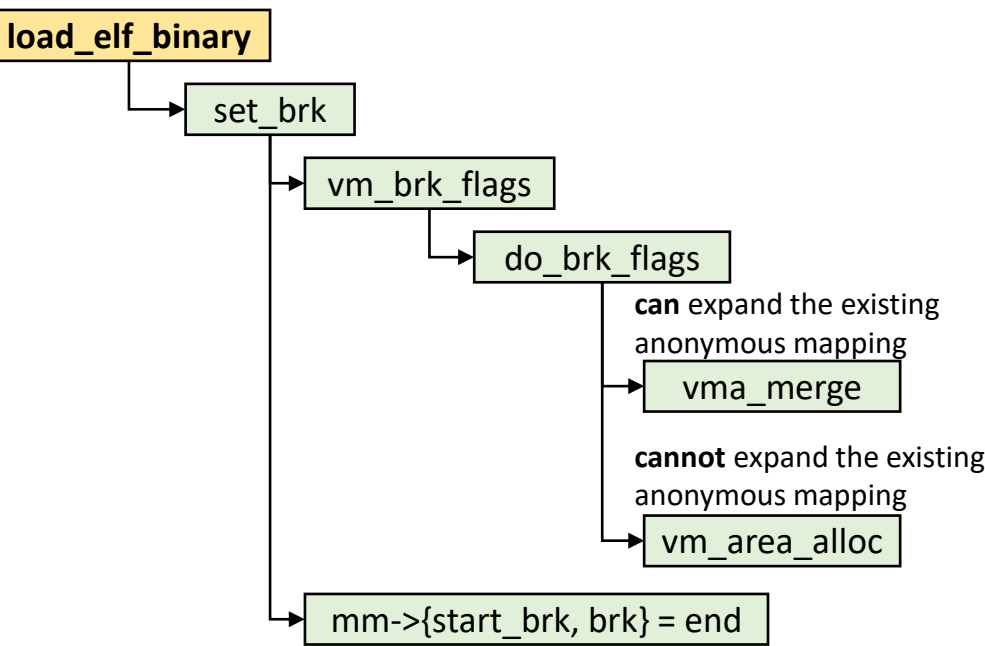
```
/ # cat /proc/44/maps
00400000-00401000 r--p 00000000 00:02 1540 /free_and_sbrk
00401000-00496000 r-xp 00001000 00:02 1540 /free_and_sbrk
00496000-004be000 r--p 00096000 00:02 1540 /free_and_sbrk
004be000-004c1000 r--p 000bd000 00:02 1540 /free_and_sbrk
004c1000-004c4000 rw-p 000c0000 00:02 1540 /free_and_sbrk
004c4000-004e8000 rw-p 00000000 00:00 0 [heap]
7ffff7ffa000-7ffff7ffe000 r--p 00000000 00:00 0 [vvar]
7ffff7ffe000-7ffff7fff000 r-xp 00000000 00:00 0 [vdso]
7ffff785d000-7ffff7fff000 rw-p 00000000 00:00 0 [stack]
ffffffff600000-ffffffff601000 --xp 00000000 00:00 0 [vsyscall]
```

Why?

GAP	vma: R	vma: R, E	vma: R	vma: R, W	GAP	vma (vvar)	vma (vdso)	GAP	vma (stack)
	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000		vm_start = 0x7ffff7ffa000	vm_start = 0x7ffff7ffe000		vm_start = 0x7ffff785d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c4000		vm_end = 0x7ffff7ffe000	vm_end = 0x7ffff7fff000		vm_end = 0x7ffff7fff000

[From scratch] Launch a program: load_elf_binary() – Heap Configuration

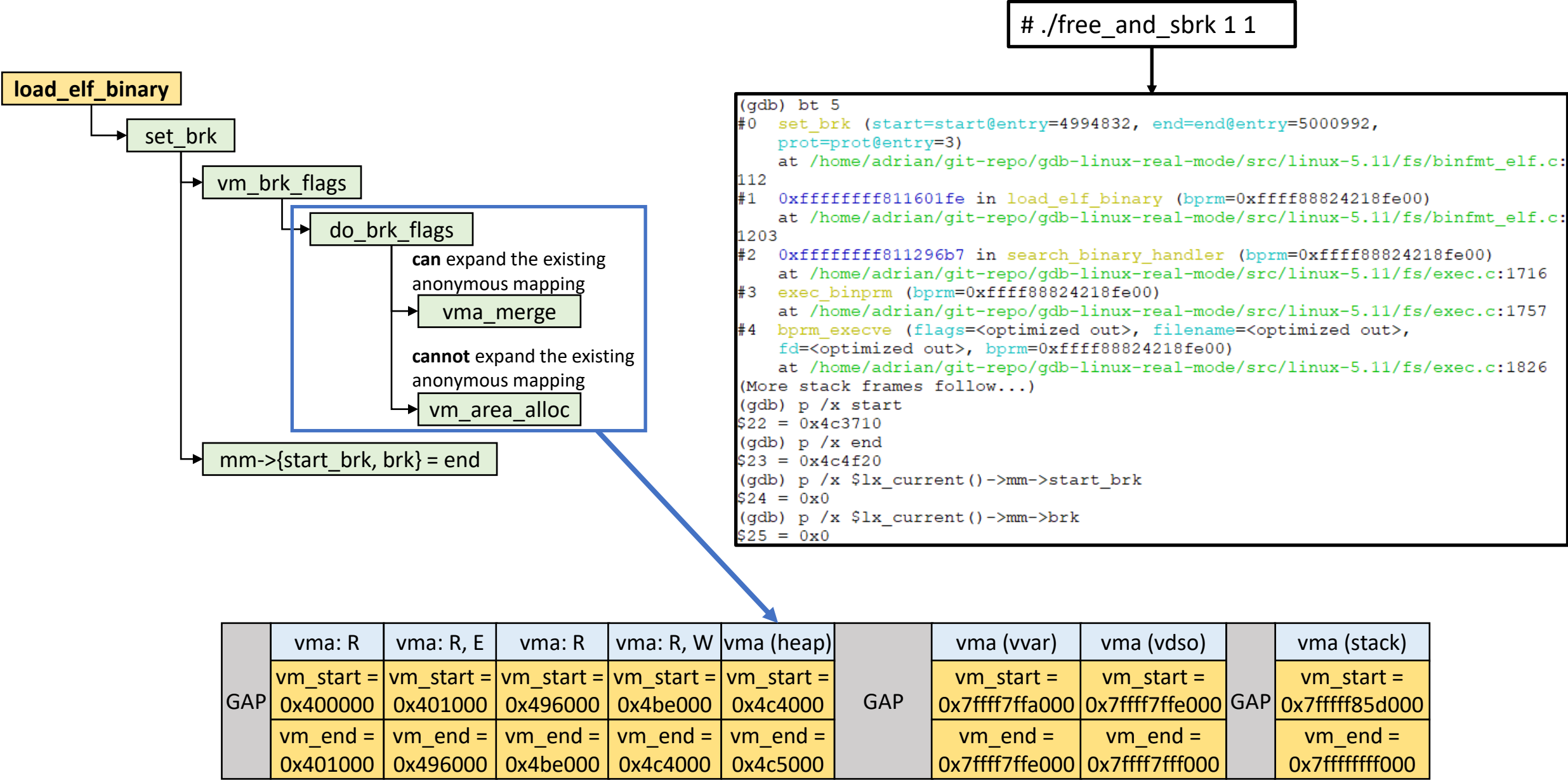
./free_and_sbrk 1 1



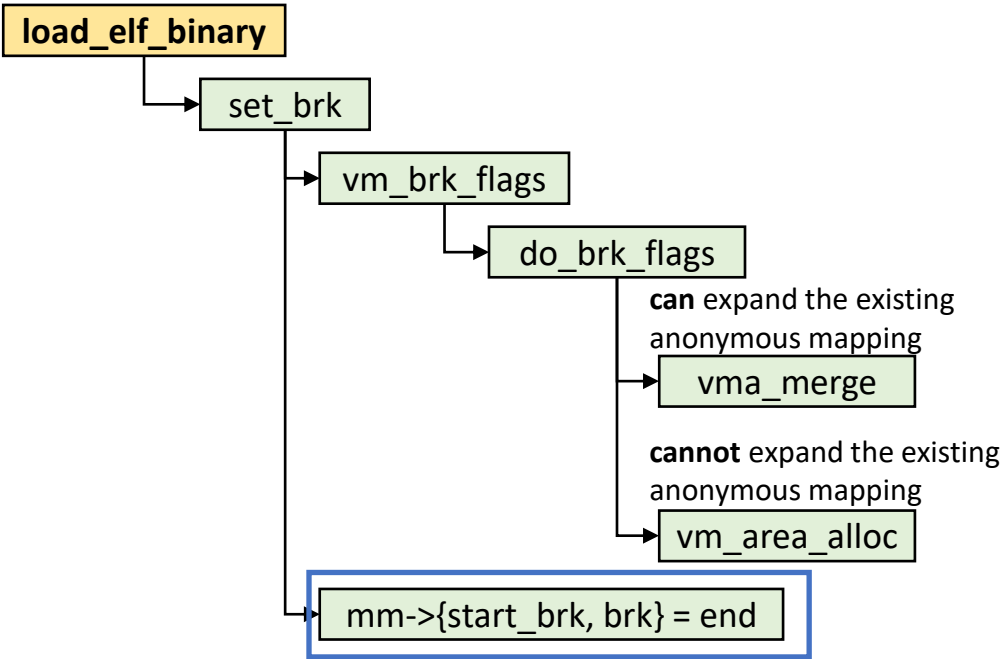
```
(gdb) bt 5
#0  set_brk (start=start@entry=4994832, end=end@entry=5000992,
    prot=prot@entry=3)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/fs/binfmt_elf.c:
112
#1  0xffffffff811601fe in load_elf_binary (bprm=0xffff88824218fe00)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/fs/binfmt_elf.c:
1203
#2  0xffffffff811296b7 in search_binary_handler (bprm=0xffff88824218fe00)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/fs/exec.c:1716
#3  exec_binprm (bprm=0xffff88824218fe00)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/fs/exec.c:1757
#4  bprm_execve (flags=<optimized out>, filename=<optimized out>,
    fd=<optimized out>, bprm=0xffff88824218fe00)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/fs/exec.c:1826
(More stack frames follow...)
(gdb) p /x start
$22 = 0x4c3710
(gdb) p /x end
$23 = 0x4c4f20
(gdb) p /x $lx_current()->mm->start_brk
$24 = 0x0
(gdb) p /x $lx_current()->mm->brk
$25 = 0x0
```

GAP	vma: R	vma: R, E	vma: R	vma: R, W	GAP	vma (vvar)	vma (vdso)	GAP	vma (stack)
	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000		vm_start = 0x7fff7ffa000	vm_start = 0x7fff7ffe000		vm_start = 0x7ffff85d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c4000		vm_end = 0x7fff7ffe000	vm_end = 0x7fff7fff000		vm_end = 0x7fffffff000

[From scratch] Launch a program: load_elf_binary() – Heap Configuration



[From scratch] Launch a program: load_elf_binary() – Heap Configuration

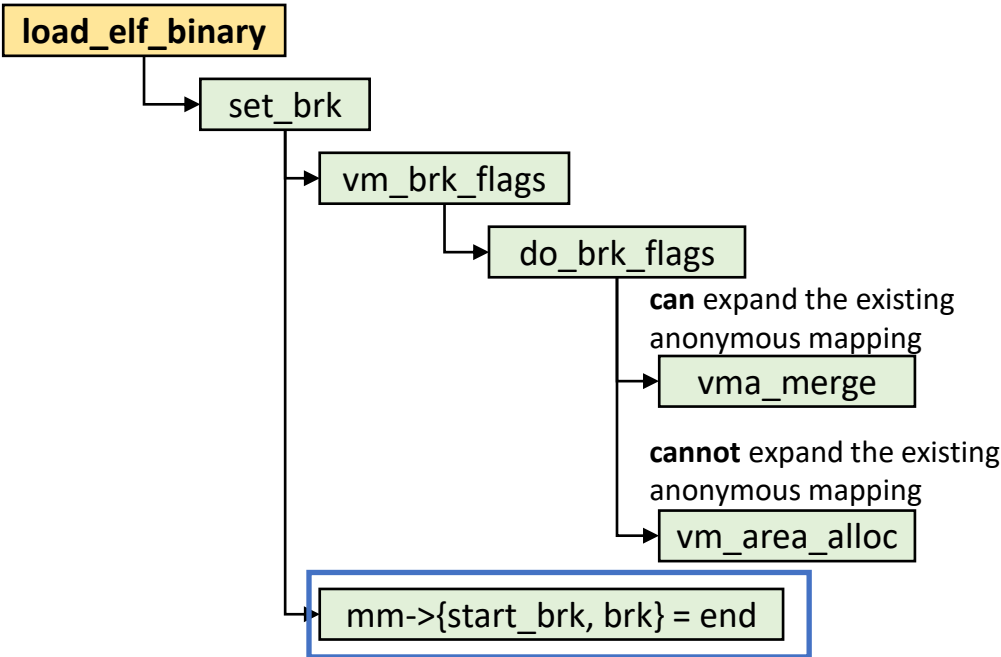


```
(gdb) bt 2
#0  set_brk (start=4997120, start@entry=4994832, end=5001216,
    end@entry=5000992, prot=prot@entry=3)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/fs/binfmt_elf.c:
126
#1  0xffffffff811601fe in load_elf_binary (bprm=0xffff8881022bea00)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/fs/binfmt_elf.c:
1203
(More stack frames follow...)
(gdb) p /x start
$16 = 0x4c4000
(gdb) p /x end
$17 = 0x4c5000
(gdb) p /x $1x_current()->mm->start_brk
$18 = 0x4c5000
(gdb) p /x $1x_current()->mm->brk
$19 = 0x4c5000
```

mm->brk = mm->start_brk
= 0x4c5000

GAP	vma: R	vma: R, E	vma: R	vma: R, W	vma (heap)	GAP	vma (vvar)	vma (vdso)	GAP	vma (stack)
	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000	vm_start = 0x4c4000		vm_start = 0x7fff7ffa000	vm_start = 0x7fff7ffe000		vm_start = 0x7ffff85d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c4000	vm_end = 0x4c5000		vm_end = 0x7fff7ffe000	vm_end = 0x7fff7fff000		vm_end = 0x7fffffff000

[From scratch] Launch a program: load_elf_binary() – Heap Configuration



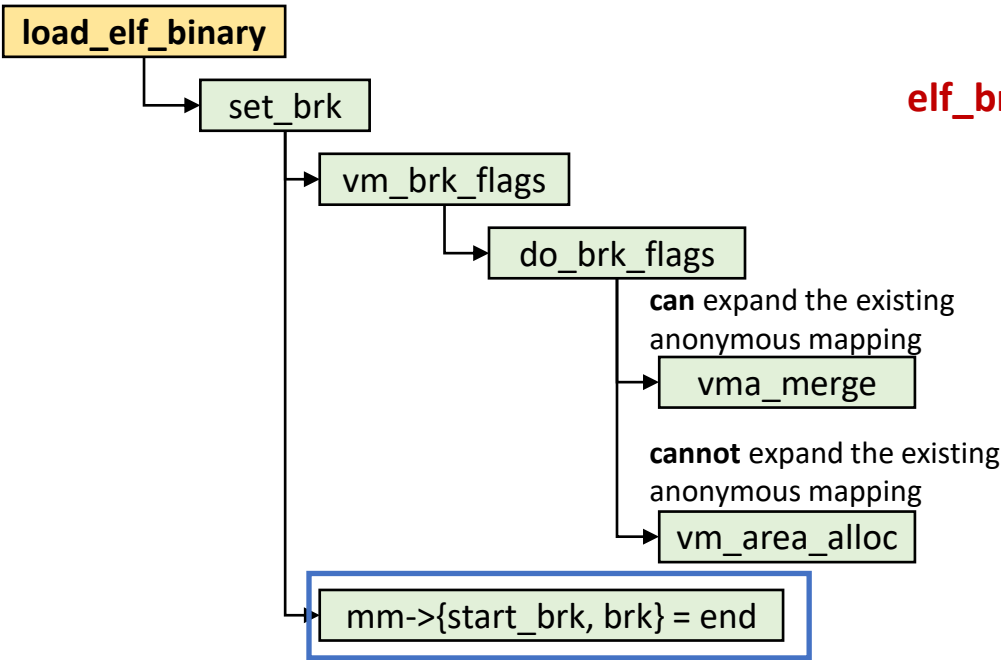
```
(gdb) bt 2
#0  set_brk (start=4997120, start@entry=4994832, end=5001216,
end@entry=5000992, prot=prot@entry=3)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/fs/binfmt_elf.c:
126
#1  0xffffffff811601fe in load_elf_binary (bprm=0xffff8881022bea00)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/fs/binfmt_elf.c:
1203
(More stack frames follow...)
(gdb) p /x start
$16 = 0x4c4000
(gdb) p /x end
$17 = 0x4c5000
(gdb) p /x $1x_current()->mm->start_brk
$18 = 0x4c5000
(gdb) p /x $1x_current()->mm->brk
$19 = 0x4c5000
```

Why?

mm->brk = mm->start_brk
= 0x4c5000

GAP	vma: R	vma: R, E	vma: R	vma: R, W	vma (heap)	GAP	vma (vvar)	vma (vdso)	GAP	vma (stack)
	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000	vm_start = 0x4c4000		vm_start = 0x7fff7ffa000	vm_start = 0x7fff7ffe000		vm_start = 0x7ffff85d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c4000	vm_end = 0x4c5000		vm_end = 0x7fff7ffe000	vm_end = 0x7fff7fff000		vm_end = 0x7fffffff000

[From scratch] Launch a program: load_elf_binary() – Heap Configuration



elf_bss

elf_brk

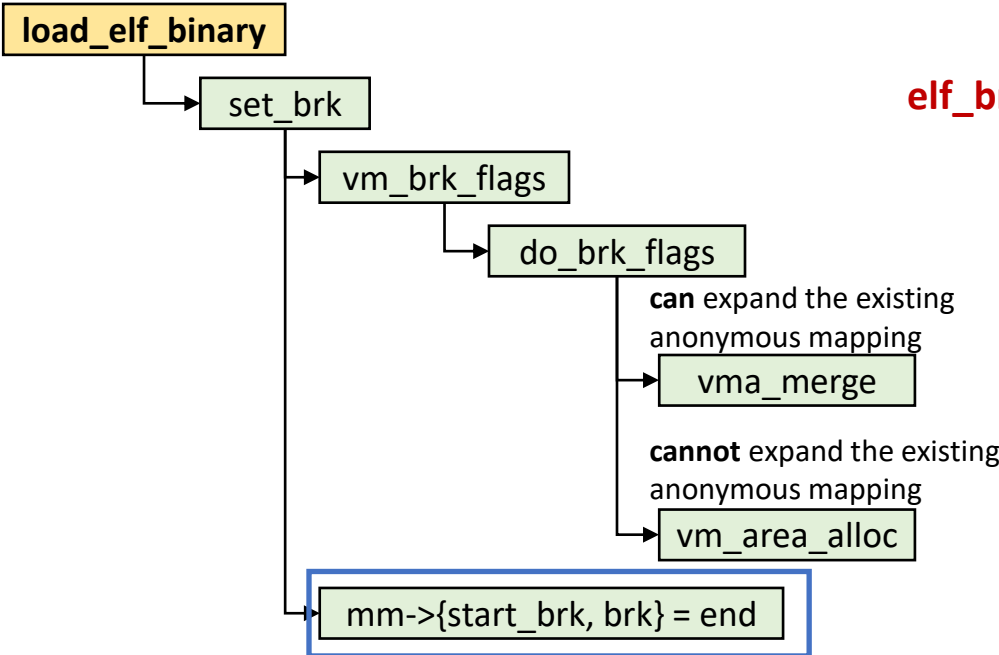
```
(gdb) bt 2
#0  set_brk (start=4997120, start@entry=4994832, end=5001216,
end@entry=5000992, prot=prot@entry=3)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/fs/binfmt_elf.c:126
#1  0xffffffff811601fe in load_elf_binary (bprm=0xffff8881022bea00)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/fs/binfmt_elf.c:1203
(More stack frames follow...)
(gdb) p /x start
$16 = 0x4c4000
(gdb) p /x end
$17 = 0x4c5000
(gdb) p /x $1x_current()->mm->start_brk
$18 = 0x4c5000
(gdb) p /x $1x_current()->mm->brk
$19 = 0x4c5000
```

```
static int load_elf_binary(struct linux_binprm *bprm)
{
+---381 lines: struct file *interpreter = NULL; to shut
    retval = set_brk(elf_bss, elf_brk, bss_prot);
fs/binfmt_elf.c
```

mm->brk = mm->start_brk
= 0x4c5000

GAP	vma: R	vma: R, E	vma: R	vma: R, W	vma (heap)	GAP	vma (vvar)	vma (vdso)	GAP	vma (stack)
	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000	vm_start = 0x4c4000		vm_start = 0x7ffff7ffa000	vm_start = 0x7ffff7ffe000		vm_start = 0x7ffff85d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c4000	vm_end = 0x4c5000		vm_end = 0x7ffff7ffe000	vm_end = 0x7ffff7fff000		vm_end = 0x7fffffff000

[From scratch] Launch a program: load_elf_binary() – Heap Configuration



elf_brk

elf_bss

```
(gdb) bt 2
#0  set_brk (start=4997120, start@entry=4994832, end=5001216,
end@entry=5000992, prot=prot@entry=3)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/fs/binfmt_elf.c:
126
#1  0xffffffff811601fe in load_elf_binary (bprm=0xfffff8881022bea00)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/fs/binfmt_elf.c:
1203
(More stack frames follow...)
(gdb) p /x start
$16 = 0x4c4000
(gdb) p /x end
$17 = 0x4c5000
(gdb) p /x $1x_current()->mm->start_brk
$18 = 0x4c5000
(gdb) p /x $1x_current()->mm->brk
$19 = 0x4c5000
```

```
static int load_elf_binary(struct linux_binprm *bprm)
{
+---381 lines: struct file *interpreter = NULL; to shut
    retval = set_brk(elf_bss, elf_brk, bss_prot);
fs/binfmt_elf.c
```

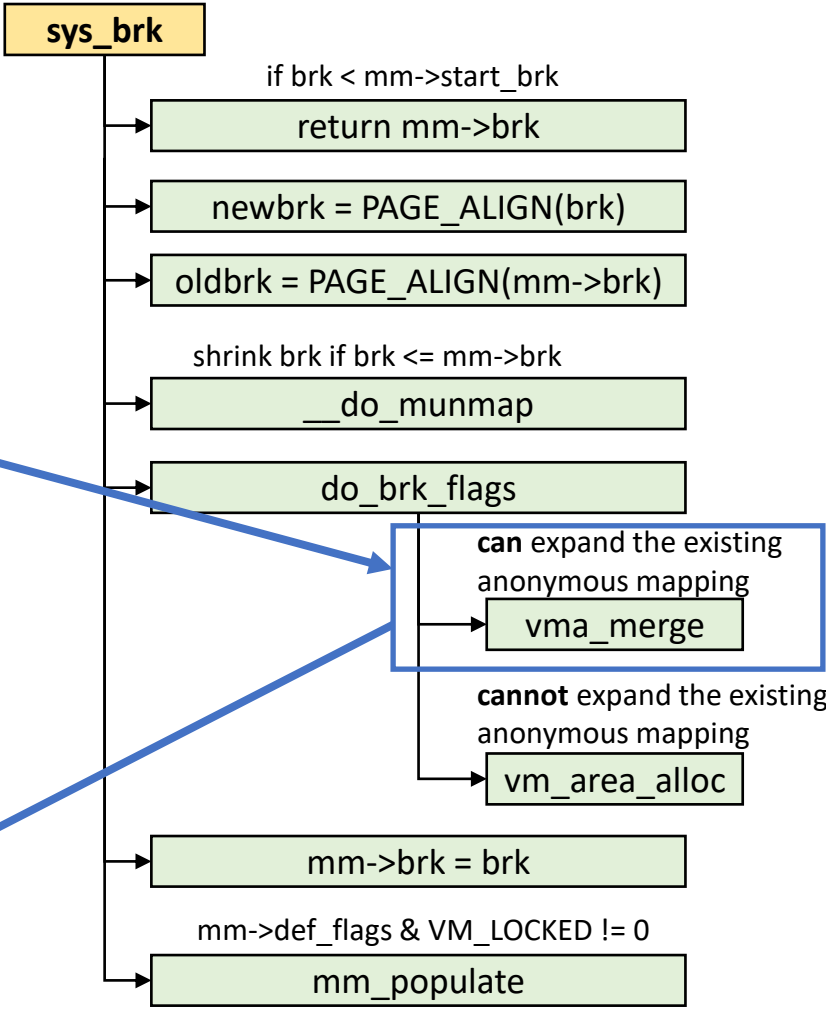
mm->brk = mm->start_brk = 0x4c5000

	vma: R	vma: R, E	vma: R	vma: R, W	vma (heap)		vma (vvar)	vma (vdso)		vma (stack)
GAP	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000	vm_start = 0x4c4000	GAP	vm_start = 0x7fff7ffa000	vm_start = 0x7fff7ffe000	GAP	vm_start = 0x7fff85d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c4000	vm_end = 0x4c5000		vm_end = 0x7fff7ffe000	vm_end = 0x7fff7fff000		vm_end = 0x7fffffff000

range(elf_bss, elf_brk): bss space

[Program Launch] strace observation: heap – brk()

```
/ # strace ./free_and_sbrk 1 1
execve("./free_and_sbrk", ["../free_and_sbrk", "1", "1"], 0x7fffffffefee30 /* 0
arch_prctl(0x3001 /* ARCH_??? */ , 0x7ffffffffffed50) = -1 EINVAL (Invalid argument)
brk(NULL) = 0x4c5000
brk(0x4c61c0) = 0x4c61c0
arch_prctl(ARCH_SET_FS, 0x4c5880) = 0
uname({sysname="Linux", nodename="(none)", ...}) = 0
readlink("/proc/self/exe", "/free_and_sbrk", 4096) = 14
brk(0x4e71c0) = 0x4e71c0
brk(0x4e8000) = 0x4e8000
mprotect(0x4be000, 12288, PROT_READ) = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x5, 0x1), ...}) = 0
ioctl(1, TCGETS, {B9600 opost -isig -icanon -echo ...}) = 0
write(1, "\n", 1) = 1
write(1, "Initial program break:      "..., 43Initial program break:      0) = 43
write(1, "Allocating 1*1 bytes\n", 21Allocating 1*1 bytes) = 21
rt_sigaction(SIGUSR1, {sa_handler=0x401d05, sa_mask=[USR1], sa_flags=SA_RESTORE0
```



mm->start_brk = 0x4c5000
mm->brk = 0x4c61c0

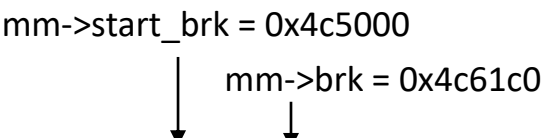
	vma: R	vma: R, E	vma: R	vma: R, W	vma (heap)		vma (vvar)	vma (vdso)		vma (stack)
GAP	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000	vm_start = 0x4c4000	GAP	vm_start = 0x7ffff7ffa000	vm_start = 0x7ffff7ffe000	GAP	vm_start = 0x7ffff85d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c4000	vm_end = 0x4c7000		vm_end = 0x7ffff7ffe000	vm_end = 0x7ffff7fff000		vm_end = 0x7ffffffffff000

Demand paging: Allocate a physical page when a page fault occurs

[Program Launch] strace observation: heap – brk()

```
/ # strace ./free_and_sbrk 1 1
execve("./free_and_sbrk", ["../free_and_sbrk", "1", "1"], 0x7fffffffefee30 /* 0
arch_prctl(0x3001 /* ARCH_??? */ , 0x7fffffffed50) = -1 EINVAL (Invalid argument)
brk(NULL) = 0x4c5000
brk(0x4c61c0) = 0x4c61c0
arch_prctl(ARCH_SET_FS, 0x4c5880) = 0
uname({sysname="Linux", nodename="(none)", ...}) = 0
readlink("/proc/self/exe", "/free_and_sbrk", 4096) = 14
brk(0x4e71c0) = 0x4e71c0
brk(0x4e8000) = 0x4e8000
mprotect(0x4be000, 12288, PROT_READ) = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x5, 0x1), ...}) = 0
ioctl(1, TCGETS, {B9600 opost -isig -icanon -echo ...}) = 0
write(1, "\n", 1
) = 1
write(1, "Initial program break:      "..., 43Initial program br
) = 43
write(1, "Allocating 1*1 bytes\n", 21Allocating 1*1 bytes
) = 21
rt_sigaction(SIGUSR1, {sa_handler=0x401d05, sa_mask=[USR1], sa_flags=
```

```
(gdb) bt 2
#0 do_brk_flags (addr=addr@entry=5001216, len=len@entry=8192, flags=1048691,
flags@entry=0, uf=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/mmap.c:3104
#1 0xffffffff810e59d7 in __do_sys_brk (brk=5005760)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/mmap.c:268
(More stack frames follow...)
(gdb) p /x 5005760
$91 = 0x4c61c0
(gdb) p /x 5001216
$92 = 0x4c5000
(gdb) p /x vma->vm_start
$93 = 0x4c4000
(gdb) p /x vma->vm_end
$94 = 0x4c7000
```

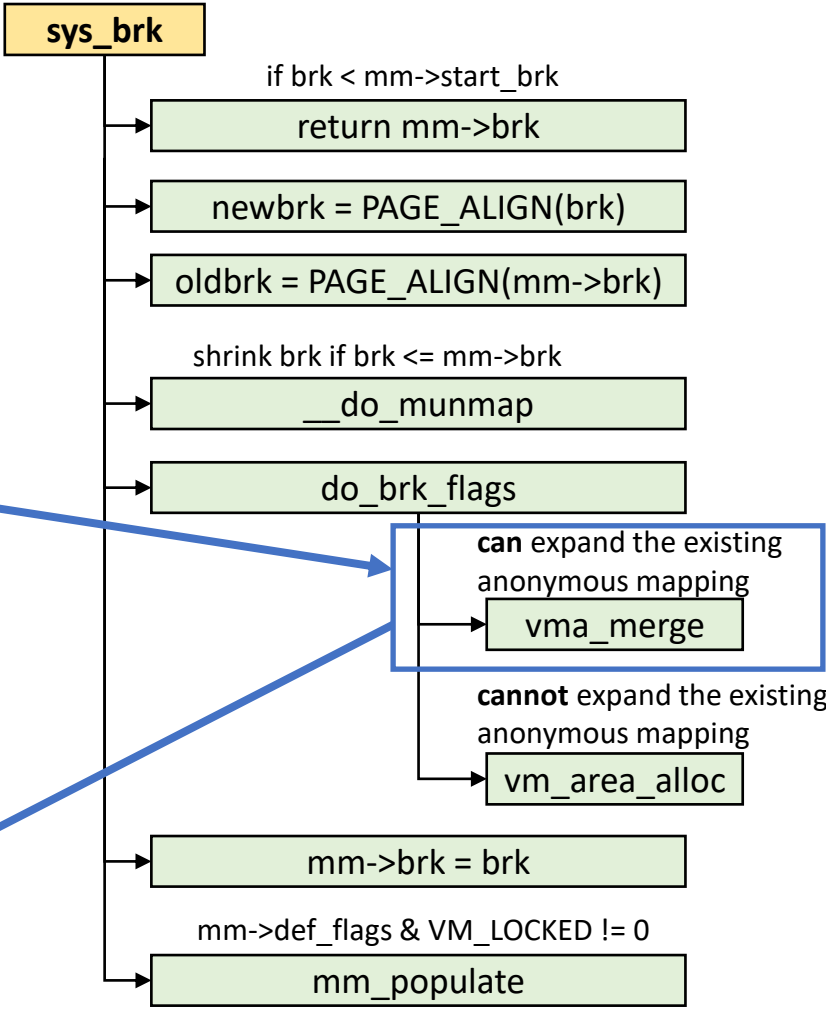


GAP	vma: R	vma: R, E	vma: R	vma: R, W	vma (heap)	GAP	vma (vvar)	vma (vdso)	GAP	vma (stack)
	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000	vm_start = 0x4c4000		vm_start = 0x7ffff7ffa000	vm_start = 0x7ffff7ffe000		vm_start = 0x7ffff85d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c4000	vm_end = 0x4c7000		vm_end = 0x7ffff7ffe000	vm_end = 0x7ffff7fff000		vm_end = 0x7ffffffffff000

Demand paging: Allocate a physical page when a page fault occurs

[Program Launch] strace observation: heap – brk()

```
/ # strace ./free_and_sbrk 1 1
execve("./free_and_sbrk", ["../free_and_sbrk", "1", "1"], 0x7fffffffefee30 /* 0
arch_prctl(0x3001 /* ARCH_??? */ , 0x7ffffffffffed50) = -1 EINVAL (Invalid argument)
brk(NULL) = 0x4c5000
brk(0x4c61c0) = 0x4c61c0
arch_prctl(ARCH_SET_FS, 0x4c5880) = 0
uname({sysname="Linux", nodename="(none)", ...}) = 0
readlink("/proc/self/exe", "/free_and_sbrk", 4096) = 14
brk(0x4e71c0) = 0x4e71c0
brk(0x4e8000) = 0x4e8000
mprotect(0x4be000, 12288, PROT_READ) = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x5, 0x1), ...}) = 0
ioctl(1, TCGETS, {B9600 opost -isig -icanon -echo ...}) = 0
write(1, "\n", 1) = 1
write(1, "Initial program break:      "..., 43Initial program break:      0) = 43
write(1, "Allocating 1*1 bytes\n", 21Allocating 1*1 bytes) = 21
rt_sigaction(SIGUSR1, {sa_handler=0x401d05, sa_mask=[USR1], sa_flags=SA_RESTORE0
```



mm->start_brk = 0x4c5000
mm->brk = 0x4e8000

	vma: R	vma: R, E	vma: R	vma: R, W	vma (heap)		vma (vvar)	vma (vdso)		vma (stack)
GAP	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000	vm_start = 0x4c4000	GAP	vm_start = 0x7ffff7ffa000	vm_start = 0x7ffff7ffe000	GAP	vm_start = 0x7ffff85d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c4000	vm_end = 0x4e8000		vm_end = 0x7ffff7ffe000	vm_end = 0x7ffff7fff000		vm_end = 0x7ffffffffff000

Demand paging: Allocate a physical page when a page fault occurs

Recap

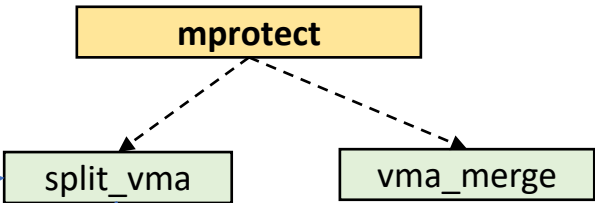
```
/ # cat /proc/44/maps
00400000-00401000 r--p 00000000 00:02 1540 /free_and_sbrk
00401000-00496000 r-xp 00001000 00:02 1540 /free_and_sbrk
00496000-004be000 r--p 00096000 00:02 1540 /free_and_sbrk
004be000-004c1000 r--p 000bd000 00:02 1540 /free_and_sbrk
004c1000-004c4000 rw-p 000c0000 00:02 1540 /free_and_sbrk
004c4000-004e8000 rw-p 00000000 00:00 0 [heap]
7ffff7ffa000-7ffff7ffe000 r--p 00000000 00:00 0 [vvar]
7ffff7ffe000-7ffff7fff000 r-xp 00000000 00:00 0 [vdso]
7ffff85d000-7ffffffffff000 rw-p 00000000 00:00 0 [stack]
ffffffff600000-ffffffff601000 --xp 00000000 00:00 0 [vsyscall]
```

Still not equal

	mm->start_brk = 0x4c5000					mm->brk = 0x4e8000				
	vma: R	vma: R, E	vma: R	vma: R, W	vma (heap)		vma (vvar)	vma (vdso)		vma (stack)
GAP	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000	vm_start = 0x4c4000	GAP	vm_start = 0x7ffff7ffa000	vm_start = 0x7ffff7ffe000	GAP	vm_start = 0x7ffff85d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c4000	vm_end = 0x4e8000		vm_end = 0x7ffff7ffe000	vm_end = 0x7ffff7fff000		vm_end = 0x7ffffffffff000

[Program Launch] strace observation: mprotect()

```
/ # strace ./free_and_sbrk 1 1
execve("./free_and_sbrk", ["/free_and_sbrk", "1", "1"], 0x7fffffff30 /* 0
arch_prctl(0x3001 /* ARCH_??? */ , 0x7fffffffed50) = -1 EINVAL (Invalid argument)
brk(NULL) = 0x4c5000
brk(0x4c61c0) = 0x4c61c0
arch_prctl(ARCH_SET_FS, 0x4c5880) = 0
uname({sysname="Linux", nodename="(none)", ...}) = 0
readlink("/proc/self/exe", "/free_and_sbrk", 4096) = 14
brk(0x4e71c0) = 0x4e71c0
brk(0x4e8000) = 0x4e8000
mprotect(0x4be000, 12288, PROT_READ) = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x5, 0x1), ...}) = 0
ioctl(1, TCGETS, {B9600 opost -isig -icanon -echo ...}) = 0
write(1, "\n", 1) = 1
write(1, "Initial program break:      "..., 43Initial program break:      0) = 43
write(1, "Allocating 1*1 bytes\n", 21Allocating 1*1 bytes) = 21
rt_sigaction(SIGUSR1, {sa_handler=0x401d05, sa_mask=[USR1], sa_flags=SA_RESTORE0
```



Split this vma

mm->start_brk = 0x4c5000
mm->brk = 0x4e8000

GAP	vma: R	vma: R, E	vma: R	vma: R, W	vma (heap)	GAP	vma (vvar)	vma (vdso)	GAP	vma (stack)
	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000	vm_start = 0x4c4000		vm_start = 0x7ffff7ffa000	vm_start = 0x7ffff7ffe000		vm_start = 0x7ffff85d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c4000	vm_end = 0x4e8000		vm_end = 0x7ffff7ffe000	vm_end = 0x7ffff7fff000		vm_end = 0x7fffffff000

[Program Launch] strace observation: mprotect()

```
/ # strace ./free_and_sbrk 1 1
execve("./free_and_sbrk", ["../free_and_sbrk", "1", "1"], 0x7fffffff30 /* 0
arch_prctl(0x3001 /* ARCH_??? */ , 0x7fffffffed50) = -1 EINVAL (Invalid argument)
brk(NULL) = 0x4c5000
brk(0x4c61c0) = 0x4c61c0
arch_prctl(ARCH_SET_FS, 0x4c5880) = 0
uname({sysname="Linux", nodename="(none)", ...}) = 0
readlink("/proc/self/exe", "/free_and_sbrk", 4096) = 14
brk(0x4e71c0) = 0x4e71c0
brk(0x4e8000) = 0x4e8000
mprotect(0x4be000, 12288, PROT_READ) = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x5, 0x1), ...}) = 0
ioctl(1, TCGETS, {B9600 opost -isig -icanon -echo ...}) = 0
write(1, "\n", 1) = 1
write(1, "Initial program break:      ...., 43Initial program break:
) = 43
write(1, "Allocating 1*1 bytes\n", 21Allocating 1*1 bytes
) = 21
rt_sigaction(SIGUSR1, {sa_handler=0x401d05, sa_mask=[USR1], sa_flags=SA_RESTORE0
```

```
#define VM_READ      0x00000001 /* currently active flags */
#define VM_WRITE     0x00000002
#define VM_EXEC      0x00000004
#define VM_SHARED     0x00000008

/* mprotect() hardcodes VM_MAYREAD >> 4 == VM_READ, and so for r/w/x bits. */
#define VM_MAYREAD   0x00000010 /* limits for mprotect() etc */
#define VM_MAYWRITE  0x00000020
#define VM_MAYEXEC   0x00000040
#define VM_MAYSHARE  0x00000080

include/linux/mm.h 249,0-1
```

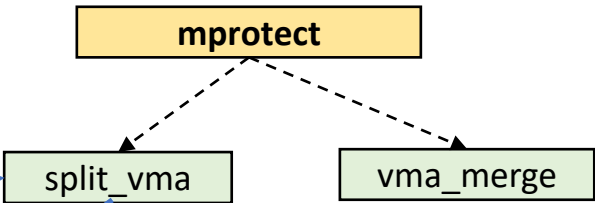
```
(gdb) p /x *$lx_current()->mm->mmap->vm_next->vm_next->vm_next
$125 = {
  vm_start = 0x4be000,
  vm_end = 0x4c4000,
  ...
  vm_flags = 0x100873, ← R/W permission
  ...
}
```

mm->start_brk = 0x4c5000
mm->brk = 0x4e8000

GAP	vma: R	vma: R, E	vma: R	vma: R, W	vma (heap)	GAP	vma (vvar)	vma (vdso)	GAP	vma (stack)
	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000	vm_start = 0x4c4000		vm_start = 0x7fff7ffa000	vm_start = 0x7fff7ffe000		vm_start = 0x7ffff85d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c4000	vm_end = 0x4e8000		vm_end = 0x7fff7ffe000	vm_end = 0x7fff7fff000		vm_end = 0x7fffffff000

[Program Launch] strace observation: mprotect()

```
/ # strace ./free_and_sbrk 1 1
execve("./free_and_sbrk", ["/free_and_sbrk", "1", "1"], 0x7fffffffef30 /* 0
arch_prctl(0x3001 /* ARCH_??? */ , 0x7fffffeded50) = -1 EINVAL (Invalid argument)
brk(NULL) = 0x4c5000
brk(0x4c61c0) = 0x4c61c0
arch_prctl(ARCH_SET_FS, 0x4c5880) = 0
uname({sysname="Linux", nodename="(none)", ...}) = 0
readlink("/proc/self/exe", "/free_and_sbrk", 4096) = 14
brk(0x4e71c0) = 0x4e71c0
brk(0x4e8000) = 0x4e8000
mprotect(0x4be000, 12288, PROT_READ) = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x5, 0x1), ...}) = 0
ioctl(1, TCGETS, {B9600 opost -isig -icanon -echo ...}) = 0
write(1, "\n", 1) = 1
write(1, "Initial program break:      "..., 43Initial program break:      0) = 43
write(1, "Allocating 1*1 bytes\n", 21Allocating 1*1 bytes) = 21
rt_sigaction(SIGUSR1, {sa_handler=0x401d05, sa_mask=[USR1], sa_flags=SA_RESTORE0
```



vma split

mm->start_brk = 0x4c5000
mm->brk = 0x4e8000

	vma: R	vma: R, E	vma: R	vma: R	vma: R, W	vma (heap)		vma (vvar)	vma (vdso)		vma (stack)
GAP	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000	vm_start = 0x4c1000	vm_start = 0x4c4000	GAP	vm_start = 0x7ffff7ffa000	vm_start = 0x7ffff7ffe000	GAP	vm_start = 0x7ffff85d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c1000	vm_end = 0x4c4000	vm_end = 0x4e8000		vm_end = 0x7ffff7ffe000	vm_end = 0x7ffff7fff000		vm_end = 0x7ffffffffff000

[Program Launch] strace observation: mprotect()

```
/ # cat /proc/44/maps
00400000-00401000 r--p 00000000 00:02 1540 /free_and_sbrk
00401000-00496000 r-xp 00001000 00:02 1540 /free_and_sbrk
00496000-004be000 r--p 00096000 00:02 1540 /free_and_sbrk
004be000-004c1000 r--p 000bd000 00:02 1540 /free_and_sbrk
004c1000-004c4000 rw-p 000c0000 00:02 1540 /free_and_sbrk
004c4000-004e8000 rw-p 00000000 00:00 0 [heap]
7ffff7ffa000-7ffff7ffe000 r--p 00000000 00:00 0 [vvar]
7ffff7ffe000-7ffff7fff000 r-xp 00000000 00:00 0 [vdso]
7ffff85d000-7ffffffffff000 rw-p 00000000 00:00 0 [stack]
ffffffff600000-ffffffff601000 --xp 00000000 00:00 0 [vsyscall]
```

match

GAP	vma: R	vma: R, E	vma: R	vma: R	vma: R, W	vma (heap)	GAP	vma (vvar)	vma (vdso)	GAP	vma (stack)
	vm_start = 0x400000	vm_start = 0x401000	vm_start = 0x496000	vm_start = 0x4be000	vm_start = 0x4c1000	vm_start = 0x4c4000		vm_start = 0x7ffff7ffa000	vm_start = 0x7ffff7ffe000		vm_start = 0x7ffff85d000
	vm_end = 0x401000	vm_end = 0x496000	vm_end = 0x4be000	vm_end = 0x4c1000	vm_end = 0x4c4000	vm_end = 0x4e8000		vm_end = 0x7ffff7ffe000	vm_end = 0x7ffff7fff000		vm_end = 0x7ffffffffff000

mm->start_brk = 0x4c5000

mm->brk = 0x4e8000

strace observation: allocate space via malloc() #1

```
/ # strace ./free_and_sbrk 1 1
execve("./free_and_sbrk", ["/free_and_sbrk", "1", "1"], 0x7fffffffefee30 /* 0
arch_prctl(0x3001 /* ARCH_??? */, 0x7fffffffd50) = -1 EINVAL (Invalid argument)
brk(NULL) = 0x4c5000
brk(0x4c61c0) = 0x4c61c0
arch_prctl(ARCH_SET_FS, 0x4c5880) = 0
uname({sysname="Linux", nodename="(none)", ...}) = 0
readlink("/proc/self/exe", "/free_and_sbrk", 4096) = 14
brk(0x4e71c0) = 0x4e71c0
brk(0x4e8000) = 0x4e8000
mprotect(0x4be000, 12288, PROT_READ) = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x5, 0x1), ...}) = 0
ioctl(1, TCGETS, {B9600 opost -isig -icanon -echo ...}) = 0
write(1, "\n", 1
) = 1
write(1, "Initial program break:      "..., 43Initial program break:      0
) = 43
write(1, "Allocating 1*1 bytes\n", 21Allocating 1*1 bytes
) = 21
rt_sigaction(SIGUSR1, {sa_handler=0x401d05, sa_mask=[USR1], sa_flags=SA_RESTORE0
```

[Init stage]
0x4e8000 – 0x4c7000 = 0x21000
(132KB: 33 pages)

[glibc] malloc

- Balance between **brk()** and **mmap()**
- Use **brk()** if request size < **DEFAULT_MMAP_THRESHOLD_MIN** (128 KB)
 - The heap can be trimmed only if memory is freed at the top end.
 - **sbrk()** is implemented as a library function that uses the **brk()** system call.
 - When the heap is used up, allocate memory chunk > 128KB via **brk()**.
 - Save overhead for frequent system call 'brk()'
- Use **mmap()** if request size >= **DEFAULT_MMAP_THRESHOLD_MIN** (128 KB)
 - The allocated memory blocks can be independently released back to the system.
 - Deallocated space is not placed on the free list for reuse by later allocations.
 - Memory may be wasted because **mmap** allocations must be page-aligned; and the kernel must perform the expensive task of zeroing out memory allocated.
 - Note: glibc uses the dynamic **mmap** threshold
 - Detail: ``man mallopt``

strace observation: allocate space via malloc() #2

malloc.c

```
#define SIZE      (128 * 1024)
#define ALLOC_SIZE (1 << 12)

int main(void)
{
    ...
    for (i = 0; i < SIZE / ALLOC_SIZE + 1; i++)
        ptr = malloc(ALLOC_SIZE);

    return 0;
}
```

[glibc] malloc

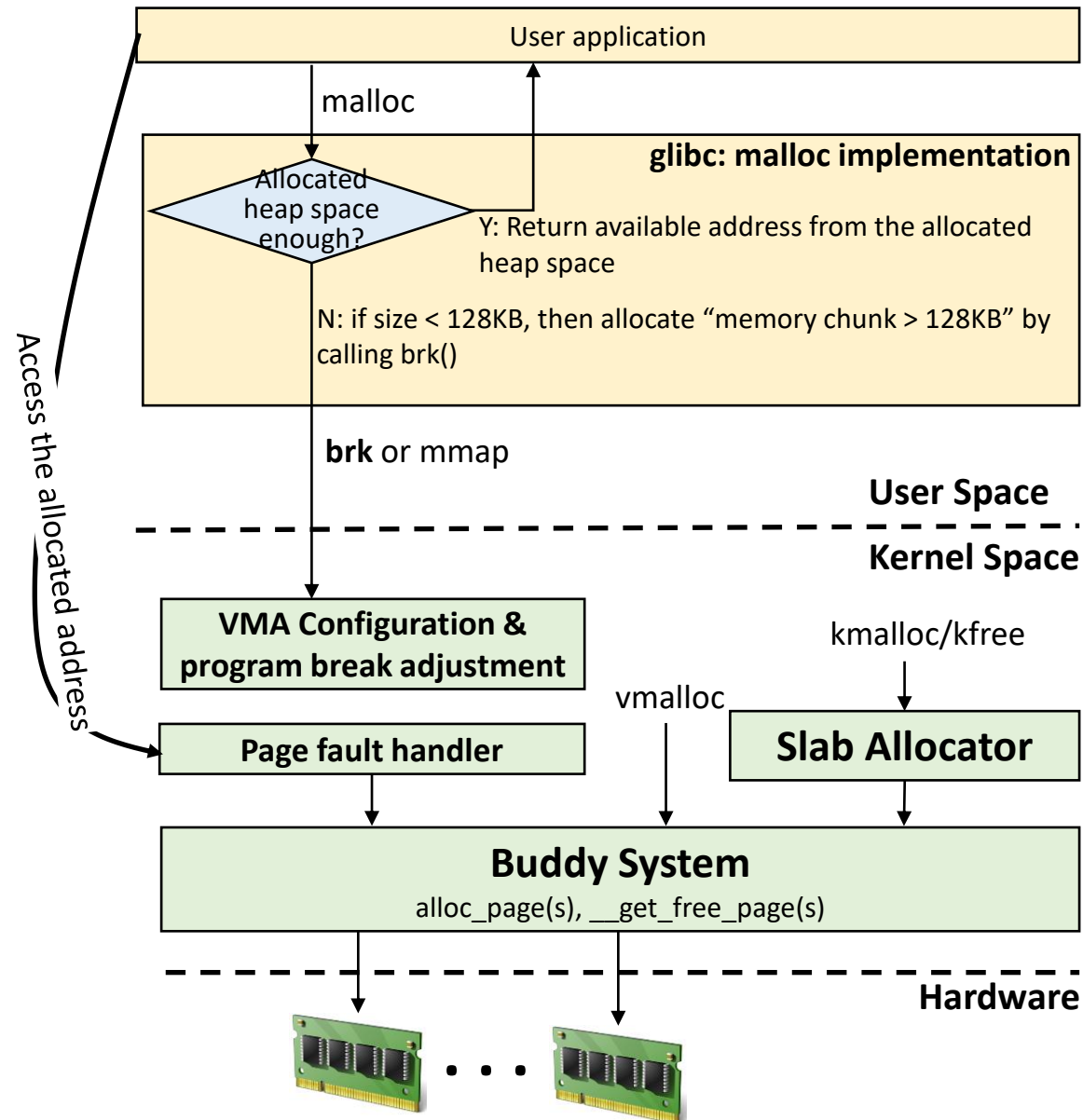
- Balance between **brk()** and **mmap()**
- Use **brk()** if request size < **DEFAULT_MMAP_THRESHOLD_MIN** (128 KB)
 - The heap can be trimmed only if memory is freed at the top end.
 - **sbrk()** is implemented as a library function that uses the **brk()** system call.
 - When the heap is used up, allocate memory chunk > 128KB via **brk()**.
 - Save overhead for frequent system call 'brk()'
- Use **mmap()** if request size >= **DEFAULT_MMAP_THRESHOLD_MIN** (128 KB)
 - The allocated memory blocks can be independently released back to the system.
 - Deallocated space is not placed on the free list for reuse by later allocations.
 - Memory may be wasted because **mmap** allocations must be page-aligned; and the kernel must perform the expensive task of zeroing out memory allocated.
 - Note: glibc uses the dynamic **mmap** threshold
 - Detail: ``man mallopt``

```
execve("./malloc", ["./malloc"], 0x7ffc74dd66d0 /* 33 vars */) = 0
brk(NULL)                                = 0x5625230c4000
...
brk(NULL)                                = 0x5625230c4000
brk(0x5625230e5000)                       = 0x5625230e5000
brk(0x562523106000)                       = 0x562523106000
exit_group(0)
+++ exited with 0 +++
```

[Init stage] 0x21000 (132KB: 33 pages)
Current program break is used
up: allocate another 132KB

Heap space allocation from malloc(): Allocate memory chunk > 128KB via brk()

Memory Allocation in Linux – brk() detail



[glibc] malloc: check sysmalloc() for implementation

- Balance between **brk()** and **mmap()**
- Use **brk()** if request size < `DEFAULT_MMAP_THRESHOLD_MIN` (128 KB)
 - The heap can be trimmed only if memory is freed at the top end.
 - `sbrk()` is implemented as a library function that uses the `brk()` system call.
 - When the heap is used up, allocate memory chunk > 128KB via `brk()`.
 - Save overhead for frequent system call 'brk()'
- Use **mmap()** if request size >= `DEFAULT_MMAP_THRESHOLD_MIN` (128 KB)
 - The allocated memory blocks can be independently released back to the system.
 - Deallocated space is not placed on the free list for reuse by later allocations.
 - Memory may be wasted because `mmap` allocations must be page-aligned; and the kernel must perform the expensive task of zeroing out memory allocated.
 - Note: glibc uses the dynamic `mmap` threshold
 - Detail: ``man mallopt``

glibc: malloc implementation for memory request size

```
adrian@adrian-ubuntu:memalloc$ gcc -o malloc malloc.c -g
adrian@adrian-ubuntu:memalloc$ gdb ./malloc -silent
Reading symbols from ./malloc...
(gdb) catch syscall brk
Catchpoint 1 (syscall 'brk' [12])
(gdb) r
Starting program: /home/adrian/archive/C/tlpi-for-qemu/tlpi-dist/memalloc/malloc

Catchpoint 1 (call to syscall brk), __brk (addr=addr@entry=0x0)
    at ../sysdeps/unix/sysv/linux/x86_64/brk.c:31
warning: Source file is more recent than executable.
31         return 0;
(gdb) bt
#0  __brk (addr=addr@entry=0x0) at ../sysdeps/unix/sysv/linux/x86_64/brk.c:31
#1  0x00007ffff7febba2 in frob_brk ()
    at ../sysdeps/unix/sysv/linux/dl-sysdep.c:36
#2  _dl_sysdep_start (start_argptr=start_argptr@entry=0x7ffffffffffe210,
    dl_main=dl_main@entry=0x7ffff7fd15e0 <dl_main>) at ../elf/dl-sysdep.c:226
#3  0x00007ffff7fd104c in _dl_start_final (arg=0x7ffffffffffe210) at rtld.c:449
#4  _dl_start (arg=0x7ffffffffffe210) at rtld.c:539
#5  0x00007ffff7fd0108 in _start () from /lib64/ld-linux-x86-64.so.2
#6  0x0000000000000001 in ?? ()
#7  0x00007ffffffffffe4a1 in ?? ()
#8  0x0000000000000000 in ?? ()
```

glibc: malloc implementation for memory request size

```
static void *
sysmalloc (INTERNAL_SIZE_T nb, mstate av)
{
+--181 lines: mchunkptr old_top;           incoming value of av->top
  else      /* av == main_arena */

    { /* Request enough space for nb + pad + overhead */
      size = nb + mp_.top_pad + MINSIZE;

+-- 16 lines: If contiguous, we can subtract out existing space that we

      size = ALIGN_UP (size, pagesize);

      /*
       Don't try to call MORECORE if argument is so big as to appear
       negative. Note that since mmap takes size_t arg, it may succeed
       below even if we cannot call MORECORE.
       */

      if (size > 0)
        {
          brk = (char *) (MORECORE (size));
          LIBC_PROBE (memory_sbrk_more, 2, brk, size);
        }

+--253 lines: if (brk != (char *) (MORECORE_FAILURE))-----
      return 0;
    }
}
malloc/malloc.c [RO] 2250,1
```

```
static struct malloc_par mp_ =
{
  .top_pad = DEFAULT_TOP_PAD,
  .n_mmaps_max = DEFAULT_MMAP_MAX,
  .mmap_threshold = DEFAULT_MMAP_THRESHOLD,
  .trim_threshold = DEFAULT_TRIM_THRESHOLD,
#define NARENAS_FROM_NCORES(n) ((n) * (sizeof (long) == 4 ? 2 : 8))
  .arena_test = NARENAS_FROM_NCORES (1)
#ifdef USE_TCACHE
  ,
  .tcache_count = TCACHE_FILL_COUNT,
  .tcache_bins = TCACHE_MAX_BINS,
  .tcache_max_bytes = tid2usize (TCACHE_MAX_BINS-1),
  .tcache_unsorted_limit = 0 /* No limit. */
#endif
};
malloc/malloc.c [RO] 1754,1
```

```
#ifndef DEFAULT_TOP_PAD
# define DEFAULT_TOP_PAD 131072
#endif
sysdeps/generic/malloc-machine.h [RO]
```

* MORECORE()->__sbrk()->__brk()

Heap space allocation from malloc(): Allocate memory chunk > 128KB via brk()

glibc: malloc implementation for memory request size

```
(gdb) c
Continuing.

Catchpoint 1 (call to syscall brk), __brk (addr=addr@entry=0x55555557a000)
  at ../sysdeps/unix/sysv/linux/x86_64/brk.c:31
31      return 0;
(gdb) bt
#0  __brk (addr=addr@entry=0x55555557a000)
  at ../sysdeps/unix/sysv/linux/x86_64/brk.c:31
#1  0x00007ffff7ec3371 in __GI__sbrk (increment=135168) at sbrk.c:56
#2  __GI__sbrk (increment=135168) at sbrk.c:31
#3  0x00007ffff7e4bb8d in __GI__default_morecore (increment=<optimized out>)
  at morecore.c:47
#4  0x00007ffff7e46575 in sysmalloc (nb=nb@entry=656,
  av=av@entry=0x7ffff7f9bb80 <main_arena>) at malloc.c:2470
#5  0x00007ffff7e47793 in _int_malloc (
  av=av@entry=0x7ffff7f9bb80 <main_arena>, bytes=bytes@entry=640)
  at malloc.c:4141
#6  0x00007ffff7e4797b in tcache_init () at malloc.c:2982
#7  0x00007ffff7e48c0e in tcache_init () at malloc.c:3044
#8  __GI__libc_malloc (bytes=4096) at malloc.c:3044
#9  malloc_hook_ini (sz=4096, caller=<optimized out>) at hooks.c:32
#10 0x0000555555555168 in main () at malloc.c:15
(gdb) f 1
#1  0x00007ffff7ec3371 in __GI__sbrk (increment=135168) at sbrk.c:56
56      sbrk.c: No such file or directory.
(gdb) p __curbrk
$8 = (void *) 0x555555559000
(gdb) f 1
#1  0x00007ffff7ec3371 in __GI__sbrk (increment=135168) at sbrk.c:56
56      in sbrk.c
(gdb) p /x increment
$9 = 0x21000
```

malloc.c

```
#define SIZE      (128 * 1024)
#define ALLOC_SIZE (1 << 12)

int main(void)
{
    ...
    for (i = 0; i < SIZE / ALLOC_SIZE + 1; i++)
        ptr = malloc(ALLOC_SIZE);

    return 0;
}
```

Detail Reference

- [\[glibc\] malloc internals](#)
 - Concept: Chunk, arenas, heaps, and thread local cache (tcache)

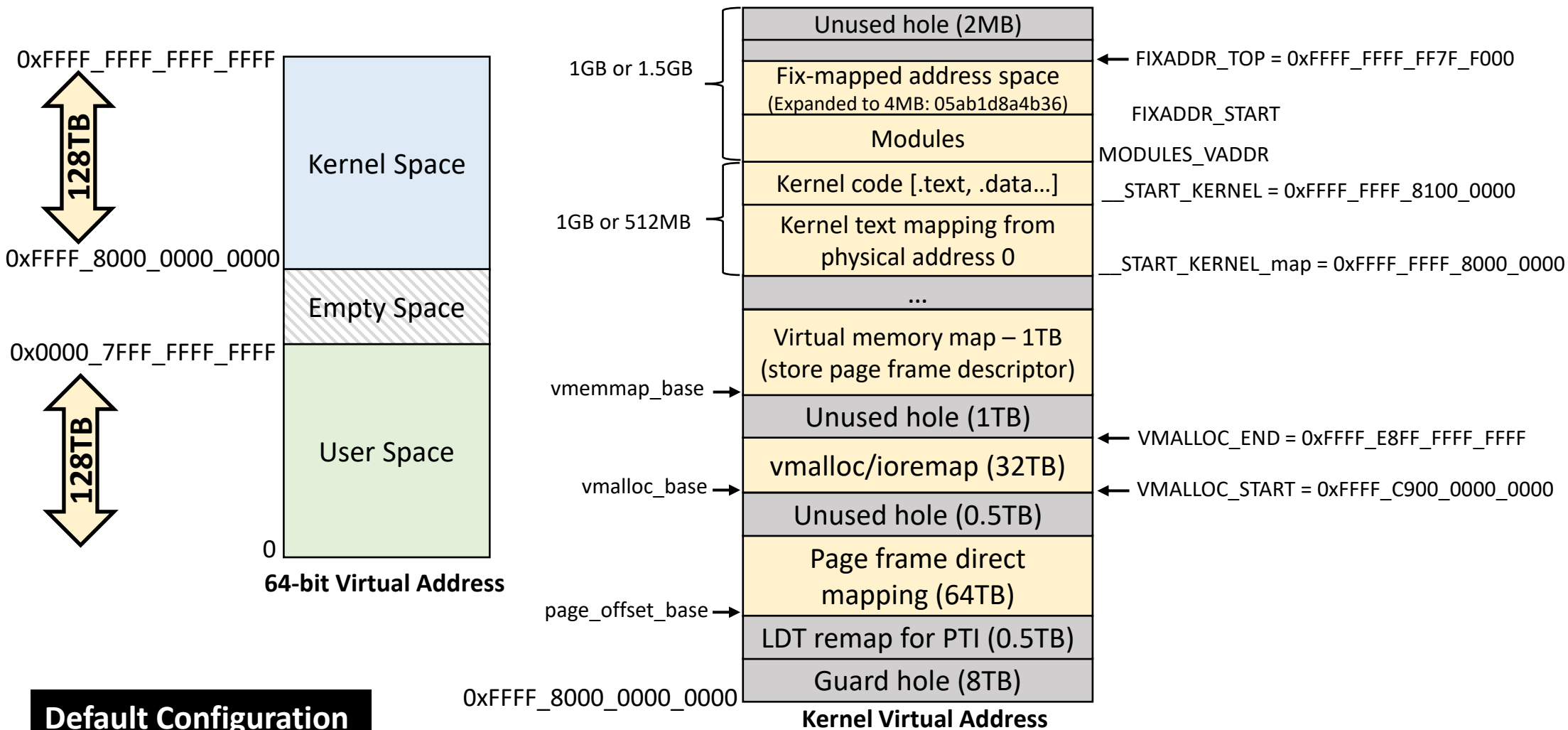
Heap is expanded for 0x21000 (33 pages): 0x555555559000 -> 0x55555557a000

vmalloc: Non-contiguous memory allocation

- 64-bit Virtual Address in x86_64
- Call path
- vmap_area & guard page
- Example: vmalloc size = 8MB
 - Kernel data structure
 - qemu + gdb observation
- vmalloc users/scenario

64-bit Virtual Address in x86_64

Reference: Documentation/x86/x86_64/mm.rst



Default Configuration

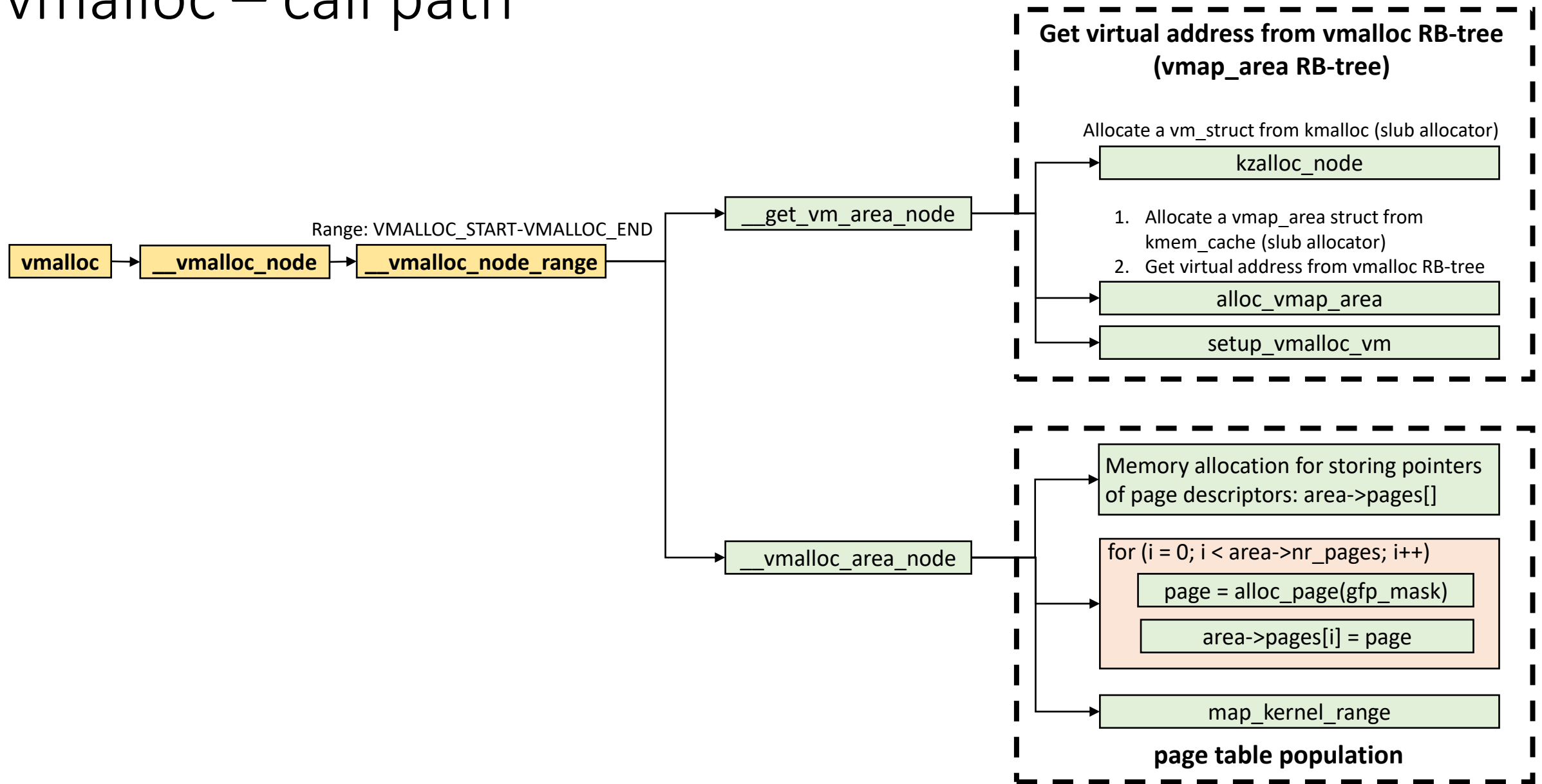
`page_offset_base = 0xFFFF_8880_0000_0000`

`vmalloc_base = 0xFFFF_C900_0000_0000`

`vmemmap_base = 0xFFFF_EA00_0000_0000`

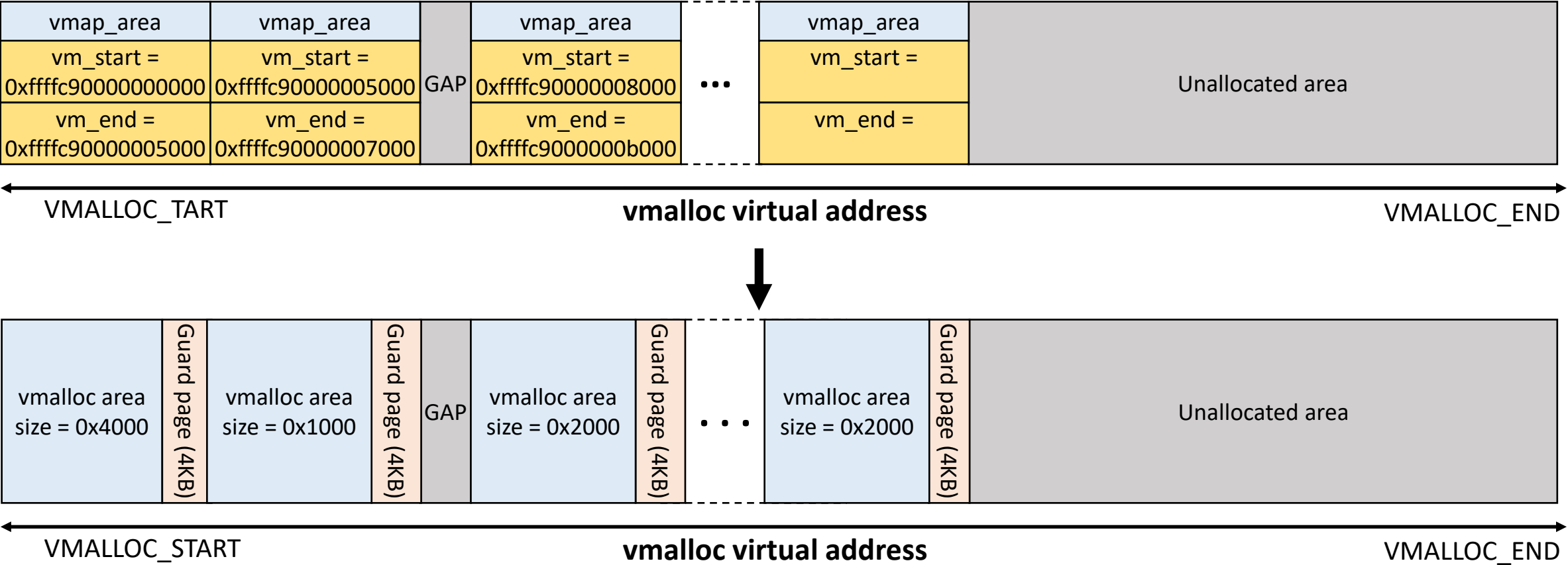
* Can be dynamically configured by KASLR (Kernel Address Space Layout Randomization - "arch/x86/mm/kaslr.c")

vmalloc – call path

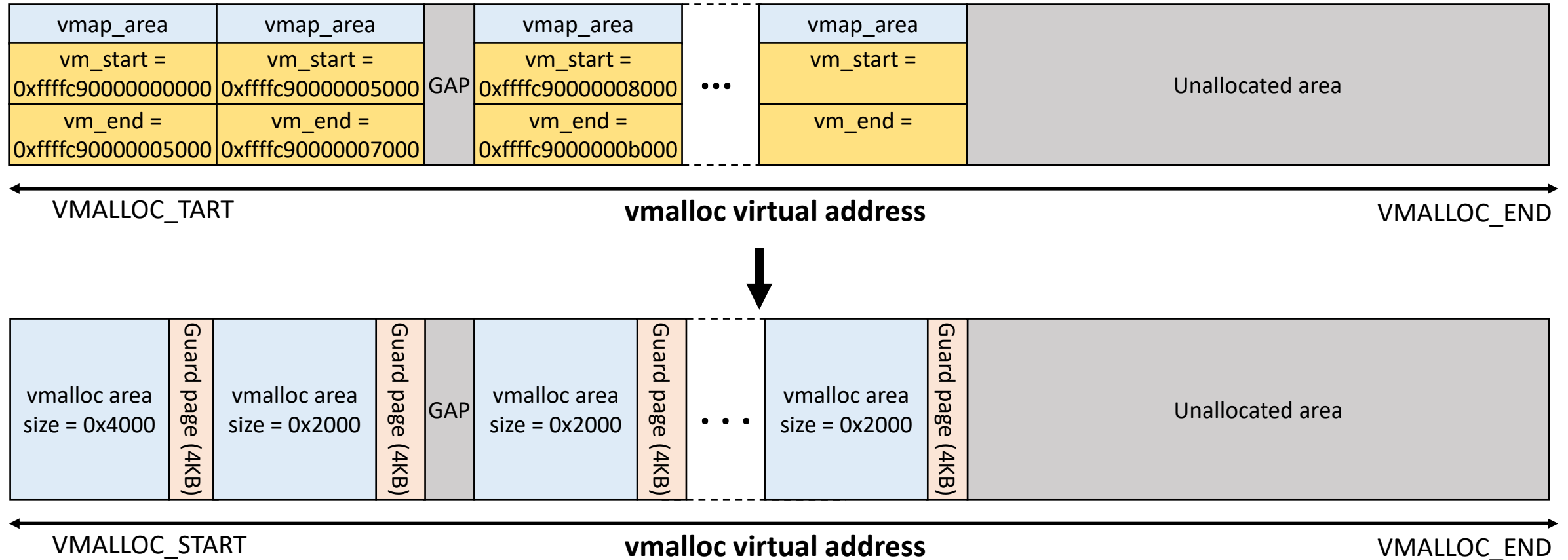


Page table is populated immediately upon the request: No page fault

vmalloc: vmap_area & guard page



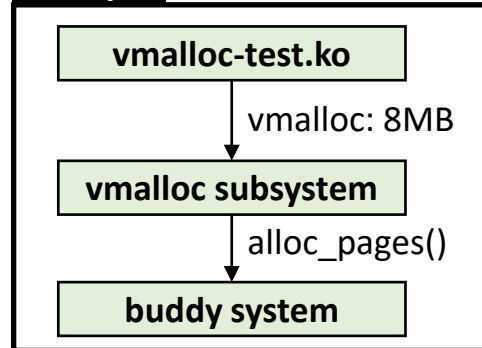
vmalloc: vmap_area & guard page



1. **Guard page (will not be allocated physically): Detect over-boundary access**
2. **VMAP_STACK kernel config: Leverage guard page (via vmalloc) to implement virtually-mapped kernel stack → Detect stack overflow**

Example: vmalloc size = 8MB: alloc_vmap_area()

Example



__vmalloc_node_range

__get_vm_area_node

__vmalloc_area_node

Get virtual address from vmalloc RB-tree (vmap_area RB-tree)

Allocate a vm_struct from kmalloc (slub allocator)

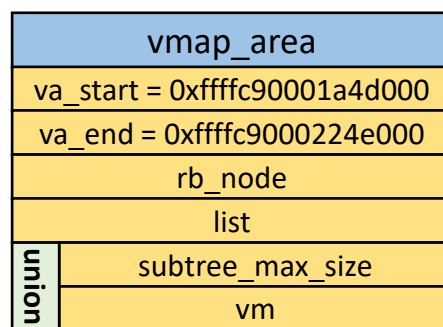
kzalloc_node

Allocate a vmap_area struct from kmem_cache (slub allocator)

alloc_vmap_area

setup_vmalloc_vm

free_vmap_area_root: init by vmalloc_init()



find_vmap_lowest_match(): Get a VA from RB-tree

insert_vmap_area()

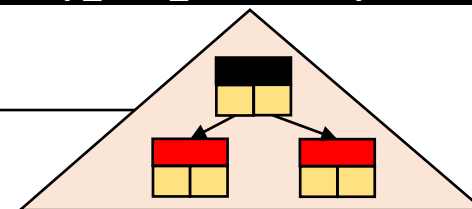
insert_vmap_area()

list_head: vmap_area_list

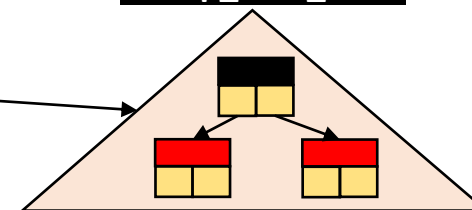
vmap_area

vmap_area

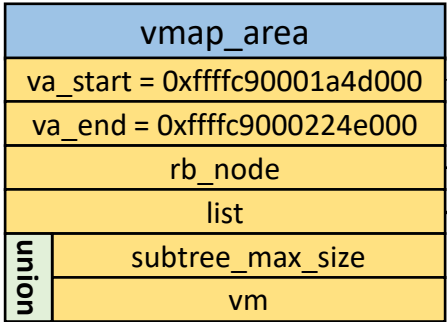
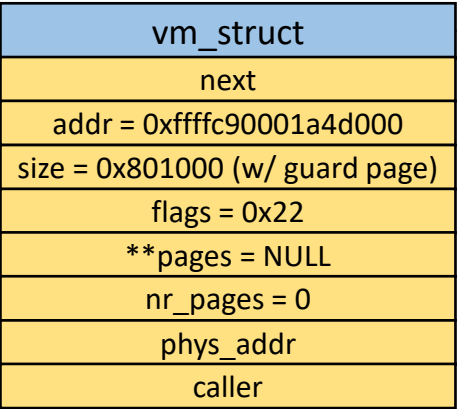
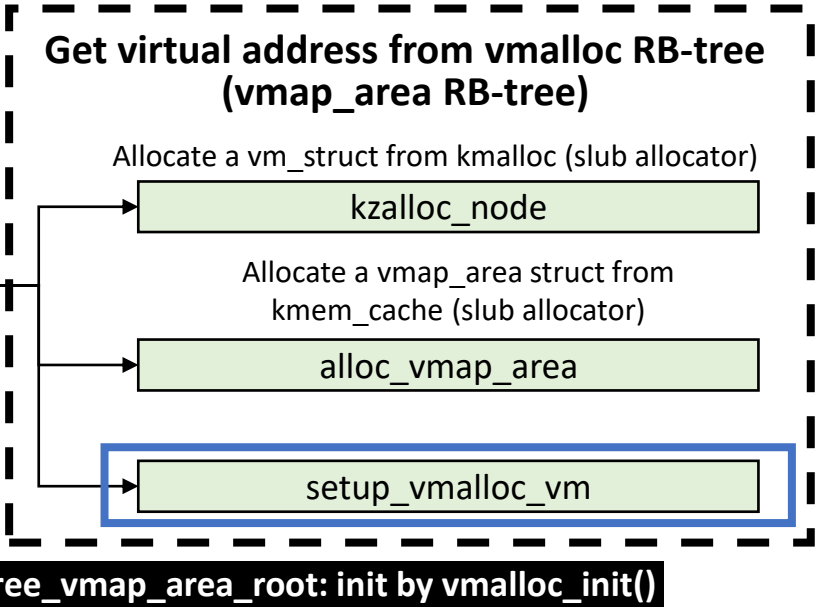
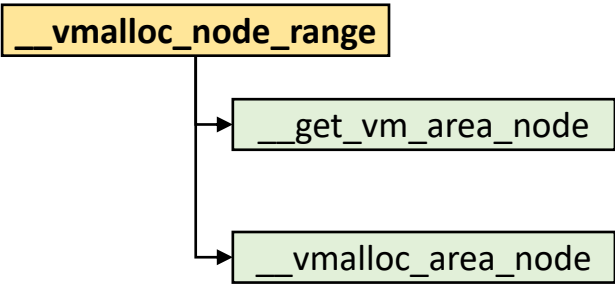
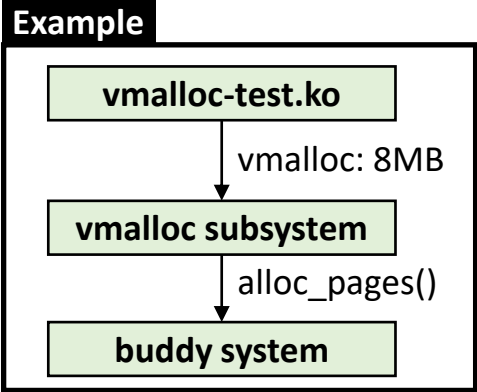
vmap_area



vmap_area_root



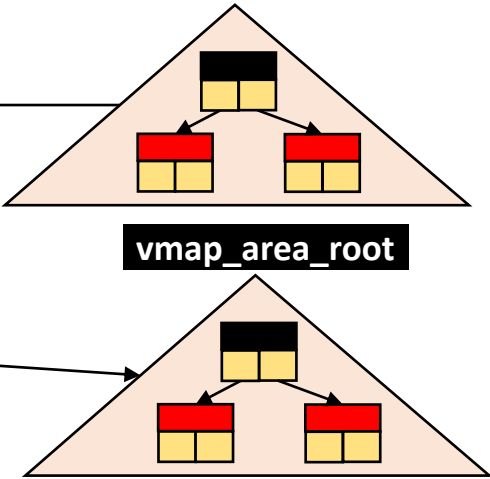
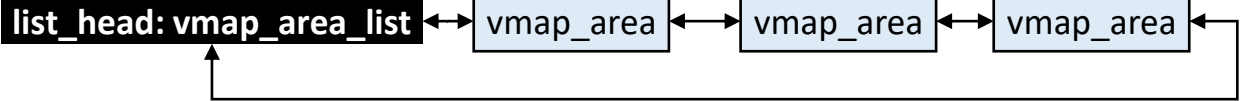
Example: vmalloc size = 8MB: setup_vmalloc_vm()



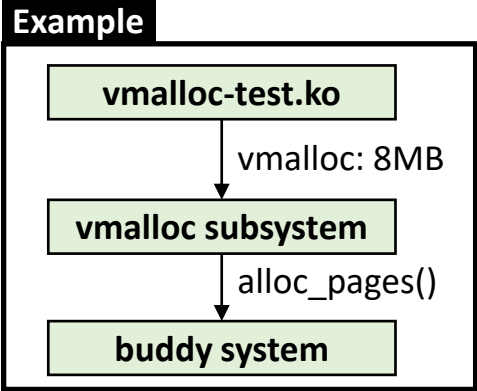
find_vmap_lowest_match(): Get a VA from RB-tree

insert_vmap_area()

insert_vmap_area()



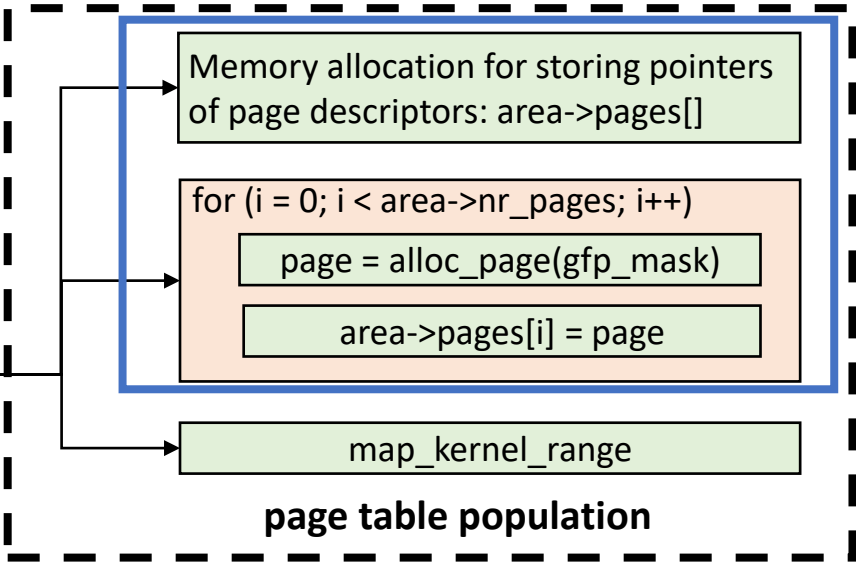
Example: vmalloc size = 8MB: `__vmalloc_area_node()`



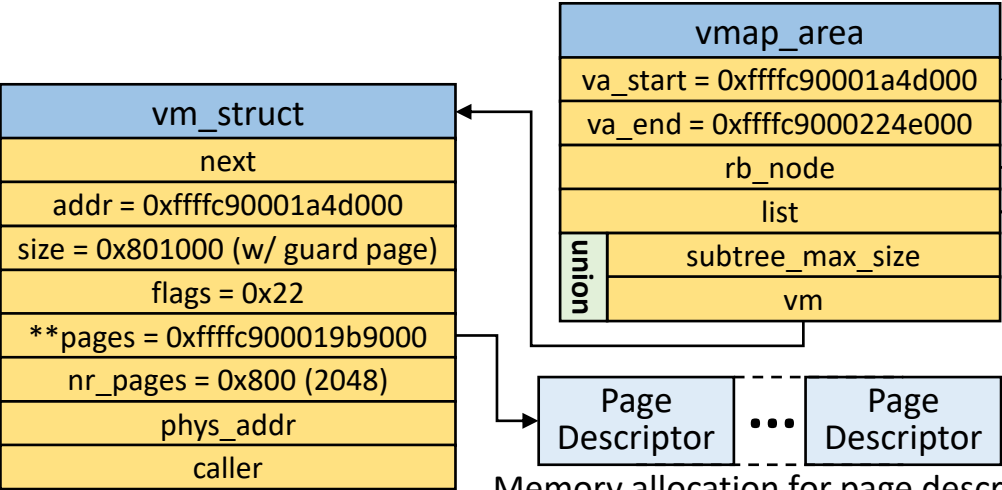
`__vmalloc_node_range`

`__get_vm_area_node`

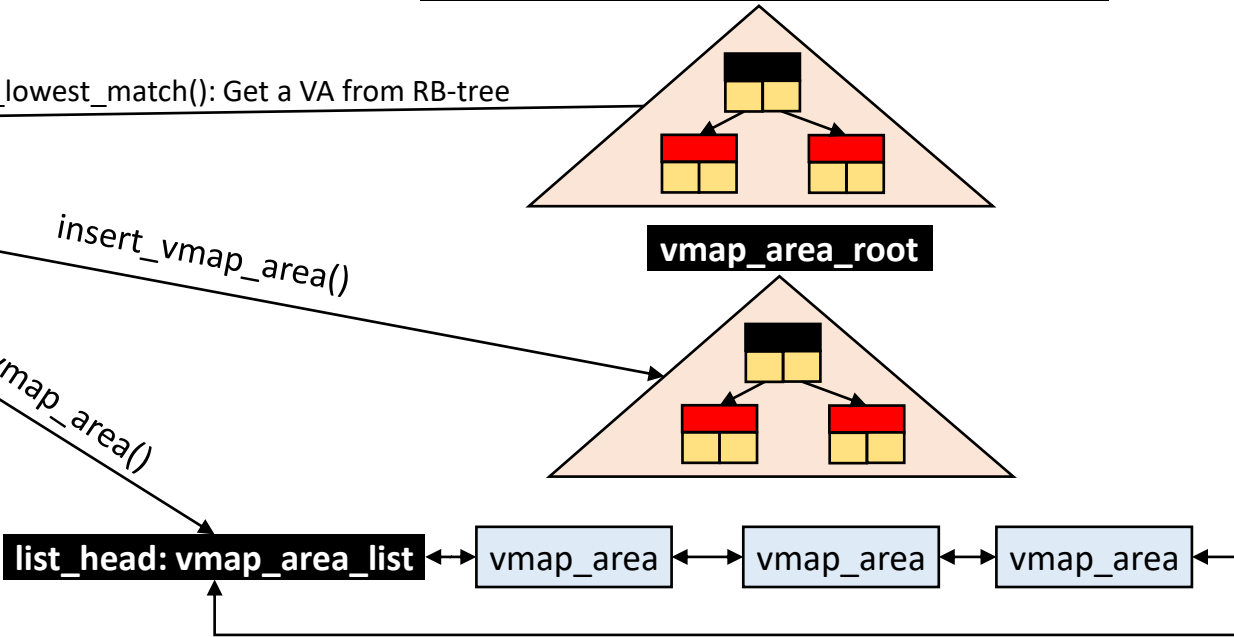
`__vmalloc_area_node`



free_vmap_area_root: init by vmalloc_init()



- Memory allocation for page descriptor pointer
- size: 8MB/4KB * 8 = 16384 bytes
 - Allocated from vmalloc (> 4KB) or kmalloc (<= 4KB)



Example: vmalloc size = 8MB

vm_struct
next
addr = 0xffffc90001a4d000
size = 0x801000 (w/ guard page)
flags = 0x22
**pages = 0xffffc900019b9000
nr_pages = 0x800 (2048)
phys_addr
caller

```
(gdb) bt 2
#0  0xfffffffff810f284b in map_kernel_range (pages=<optimized out>, prot=...,
      size=<optimized out>, start=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/include/linux/vm
      alloc.h:150
#1  __vmalloc_area_node (node=-1, prot=..., gfp_mask=11458,
      area=0xfffff8881021dc580)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:252
6
(More stack frames follow...)
(gdb) f 1
#1  __vmalloc_area_node (node=-1, prot=..., gfp_mask=11458,
      area=0xfffff8881021dc580)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:252
6
2526         if (map_kernel_range((unsigned long)area->addr, get_vm_area_size
      (area),
(gdb) p area->pages
$6 = (struct page **) 0xffffc900019b9000
(gdb) x /16g area->pages
0xffffc900019b9000: 0xfffffea0004090680 0xfffffea00040906c0
0xffffc900019b9010: 0xfffffea0004090700 0xfffffea0004090740
0xffffc900019b9020: 0xfffffea0004090780 0xfffffea00040907c0
0xffffc900019b9030: 0xfffffea0004090000 0xfffffea0004090040
0xffffc900019b9040: 0xfffffea0004090080 0xfffffea00040900c0
0xffffc900019b9050: 0xfffffea0004090100 0xfffffea0004090140
0xffffc900019b9060: 0xfffffea0004090180 0xfffffea00040901c0
0xffffc900019b9070: 0xfffffea0004090200 0xfffffea0004090240
```

6 contiguous pages

N contiguous pages

vmalloc: Virtually contiguous address; Physically non-contiguous address

Example: vmalloc size = 8MB

vm_struct
next
addr = 0xffffc90001a4d000
size = 0x801000 (w/ guard page)
flags = 0x22
**pages = 0xffffc900019b9000
nr_pages = 0x800 (2048)
phys_addr
caller

```
(gdb) bt 2
#0  0xfffffffff810f284b in map_kernel_range (pages=<optimized out>, prot=...,
      size=<optimized out>, start=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/include/linux/vm
      alloc.h:150
#1  __vmalloc_area_node (node=-1, prot=..., gfp_mask=11458,
      area=0xfffff8881021dc580)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:252
6
(More stack frames follow...)
(gdb) f 1
#1  __vmalloc_area_node (node=-1, prot=..., gfp_mask=11458,
      area=0xfffff8881021dc580)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:252
6
2526             if (map_kernel_range((unsigned long)area->addr, get_vm_area_size
      (area),
(gdb) p area->pages
$6 = (struct page **) 0xffffc900019b9000
(gdb) x /16g area->pages
0xffffc900019b9000:  0xfffffea0004090680      0xfffffea00040906c0
0xffffc900019b9010:  0xfffffea0004090700      0xfffffea0004090740
0xffffc900019b9020:  0xfffffea0004090780      0xfffffea00040907c0
0xffffc900019b9030:  0xfffffea0004090000      0xfffffea0004090040
0xffffc900019b9040:  0xfffffea0004090080      0xfffffea00040900c0
0xffffc900019b9050:  0xfffffea0004090100      0xfffffea0004090140
0xffffc900019b9060:  0xfffffea0004090180      0xfffffea00040901c0
0xffffc900019b9070:  0xfffffea0004090200      0xfffffea0004090240
```

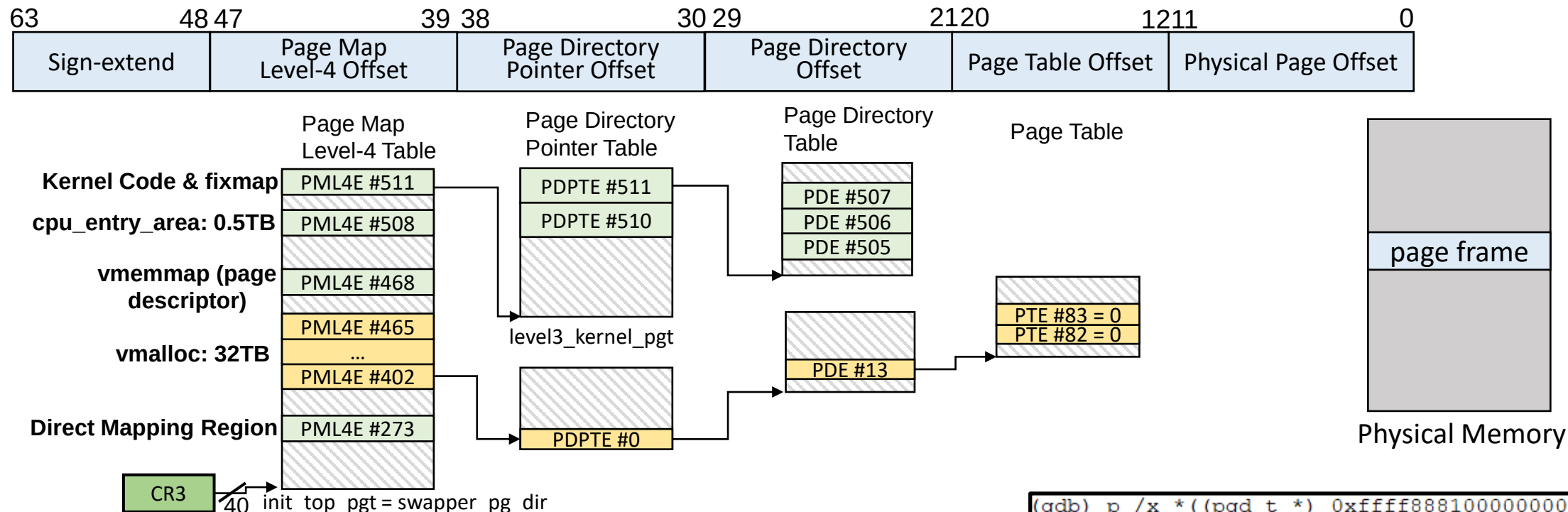
```
(gdb) p *area
$7 = {
  next = 0x0 <fixed_percpu_data>,
  addr = 0xffffc90001a4d000,
  size = 8392704,
  flags = 34,
  pages = 0xffffc900019b9000,
  nr_pages = 2048,
  phys_addr = 0,
  caller = 0xffffffffc0000010
}
```

vmalloc virtual address:
0xffff_c900_01a5_3000

vmalloc virtual address:
0xffff_c900_01a4_d000 + 0x5000 =
0xffff_c900_01a5_2000

Example: vmalloc size = 8MB: Page Table Configuration

[Linear Address] 0xffff_c900_01a5_2000, 0xffff_c900_01a5_3000

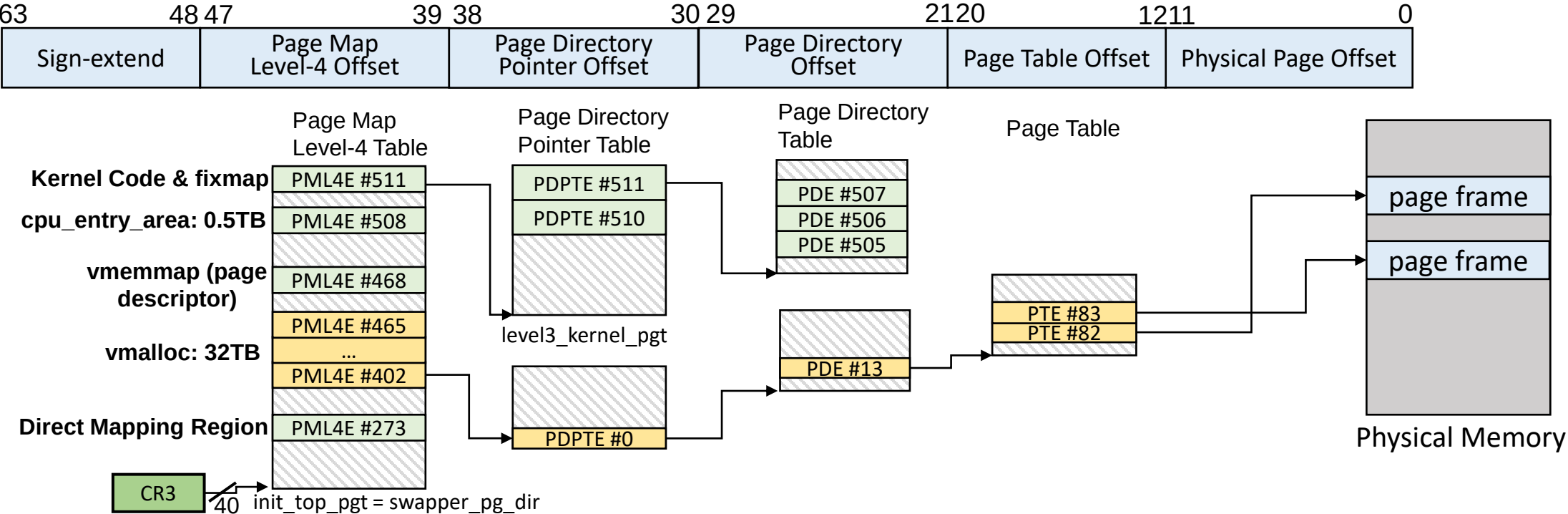


```
(gdb) bt 1
#0 0xffffffff810f284b in map_kernel_range (pages=<optimized out>, prot=...,
size=<optimized out>, start=<optimized out>)
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/include/linux/vm
alloc.h:150
(More stack frames follow...)
(gdb) p $lx_current()->mm->pgd
$9 = (pgd_t *) 0xffff8881021e8000
(gdb) p /x *((pgd_t *) 0xffff8881021e8000 + 402)
$10 = {
  pgd = 0x100000067
}
```

```
(gdb) p /x *((pgd_t *) 0xffff888100000000 + 0)
$13 = {
  pgd = 0x10005a067
}
(gdb) p /x *((pgd_t *) 0xffff88810005a000 + 13)
$14 = {
  pgd = 0x1021b7067
}
(gdb) p /x *((pgd_t *) 0xffff8881021b7000 + 82)
$15 = {
  pgd = 0x0
}
(gdb) p /x *((pgd_t *) 0xffff8881021b7000 + 83)
$16 = {
  pgd = 0x0
}
```

Example: vmalloc size = 8MB: Page Table Configuration

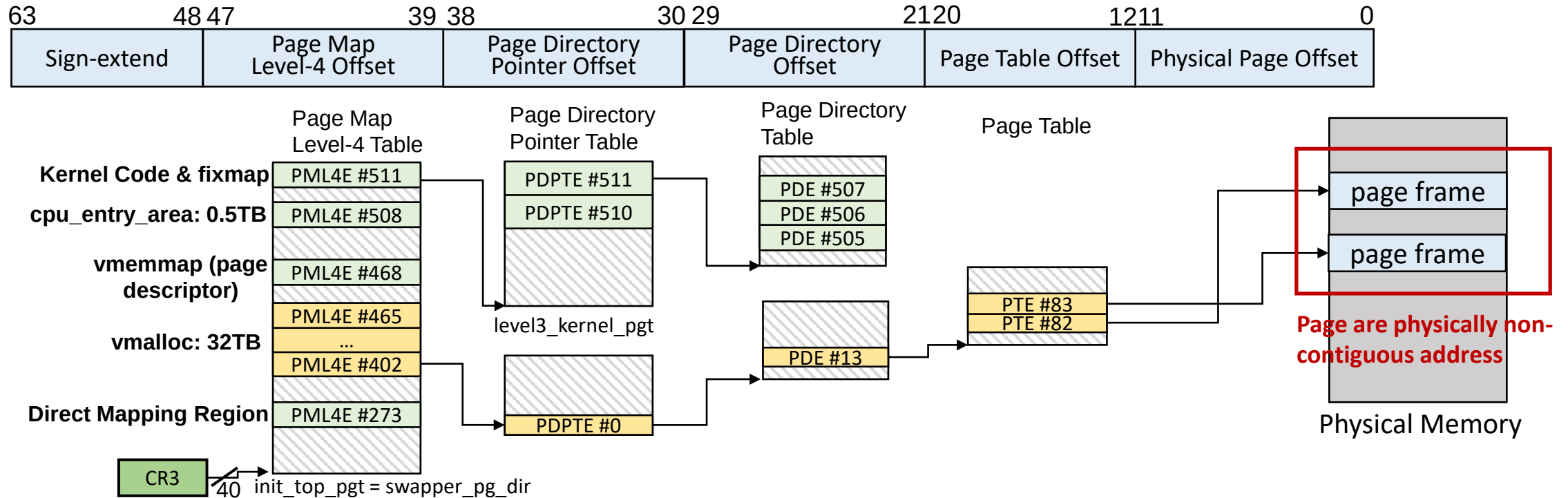
[Linear Address] 0xffff_c900_01a5_2000, 0xffff_c900_01a5_3000



```
(gdb) bt 1
#0  __vmalloc_area_node (node=-1, prot=..., gfp_mask=11458,
    area=0xffff8881021dc580)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:253
0
(More stack frames follow...)
(gdb) p /x *((pgd_t *) 0xffff8881021b7000 + 82)
$17 = {
    pgd = 0x8000000010241f063
}
(gdb) p /x *((pgd_t *) 0xffff8881021b7000 + 83)
$18 = {
    pgd = 0x80000000102400063
}
```

Example: vmalloc size = 8MB: Page Table Configuration

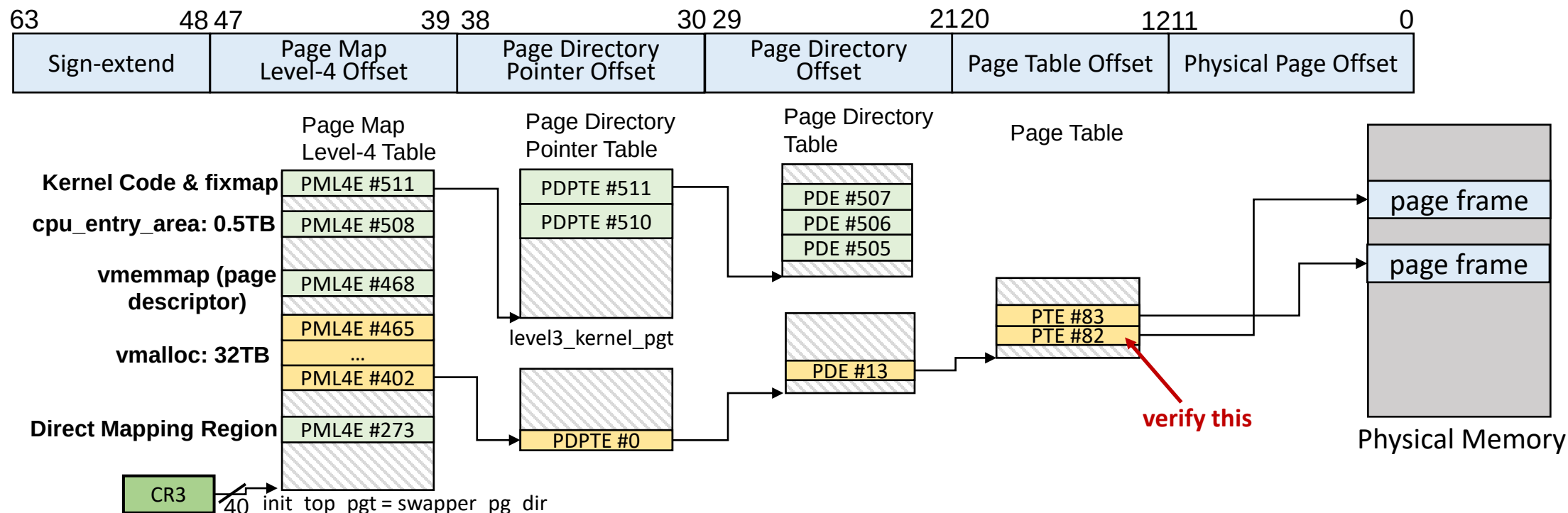
[Linear Address] 0xffff_c900_01a5_2000, 0xffff_c900_01a5_3000



```
(gdb) bt 1
#0  __vmalloc_area_node (node=-1, prot=..., gfp_mask=11458,
    area=0xffff8881021dc580)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:253
0
(More stack frames follow...)
(gdb) p /x *((pgd_t *) 0xffff8881021b7000 + 82)
$17 = {
    pgd = 0x8000000010241f063
}
(gdb) p /x *((pgd_t *) 0xffff8881021b7000 + 83)
$18 = {
    pgd = 0x80000000102400063
}
```

Example: vmalloc size = 8MB: Page Table Configuration

[Linear Address] 0xffff_c900_01a5_2000, 0xffff_c900_01a5_3000



```
(gdb) bt 1
#0  __vmalloc_area_node (node=-1, prot=..., gfp_mask=11458,
    area=0xffff8881021dc580)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:253
```

(More stack frames follow...)

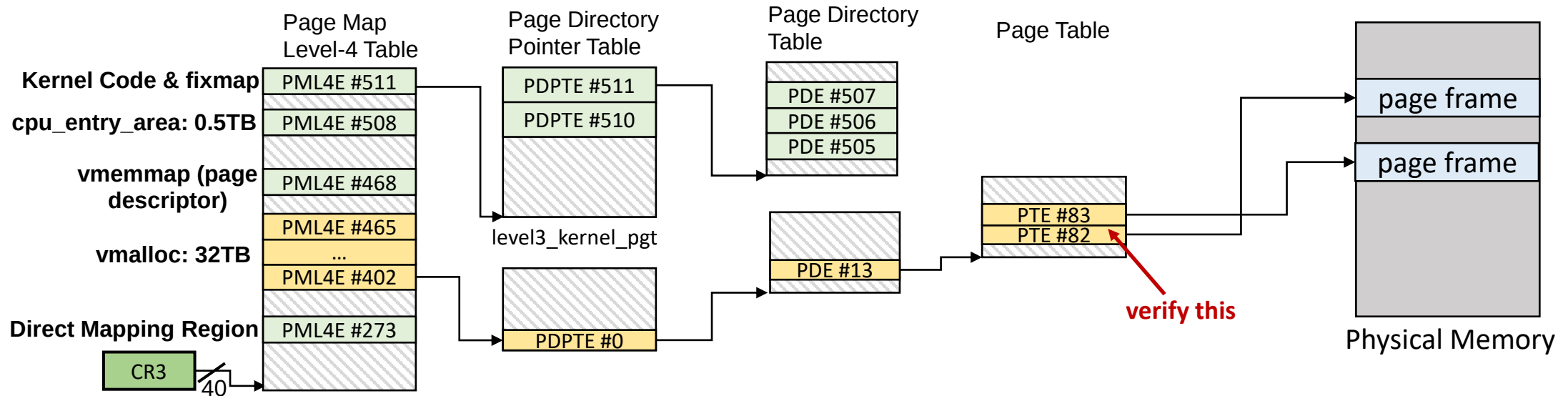
```
(gdb) p /x *((pgd_t *) 0xffff8881021b7000 + 82)
$17 = {
    pgd = 0x8000000010241f063
}
```

```
(gdb) p /x *((pgd_t *) 0xffff8881021b7000 + 83)
$18 = {
    pgd = 0x80000000102400063
}
```

```
(gdb) x /16g area->pages
```

0xfffffc900019b9000:	0xfffffea0004090680	0xfffffea00040906c0
0xfffffc900019b9010:	0xfffffea0004090700	0xfffffea0004090740
0xfffffc900019b9020:	0xfffffea0004090780	0xfffffea00040907c0
0xfffffc900019b9030:	0xfffffea0004090000	0xfffffea0004090040
0xfffffc900019b9040:	0xfffffea0004090080	0xfffffea00040900c0
0xfffffc900019b9050:	0xfffffea0004090100	0xfffffea0004090140
0xfffffc900019b9060:	0xfffffea0004090180	0xfffffea00040901c0
0xfffffc900019b9070:	0xfffffea0004090200	0xfffffea0004090240

Example: vmalloc size = 8MB: Page Table Configuration



```
(gdb) bt 1
#0  __vmalloc_area_node (node=-1, prot=..., gfp_mask=11458,
    area=0xffff8881021dc580)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:253
0
```

(More stack frames follow...)

```
(gdb) p /x *((pgd_t *) 0xffff8881021b7000 + 82)
```

```
$17 = {
    pgd = 0x800000010241f063
}
```

```
(gdb) p /x *((pgd_t *) 0xffff8881021b7000 + 83)
```

```
$18 = {
    pgd = 0x8000000102400063
}
```

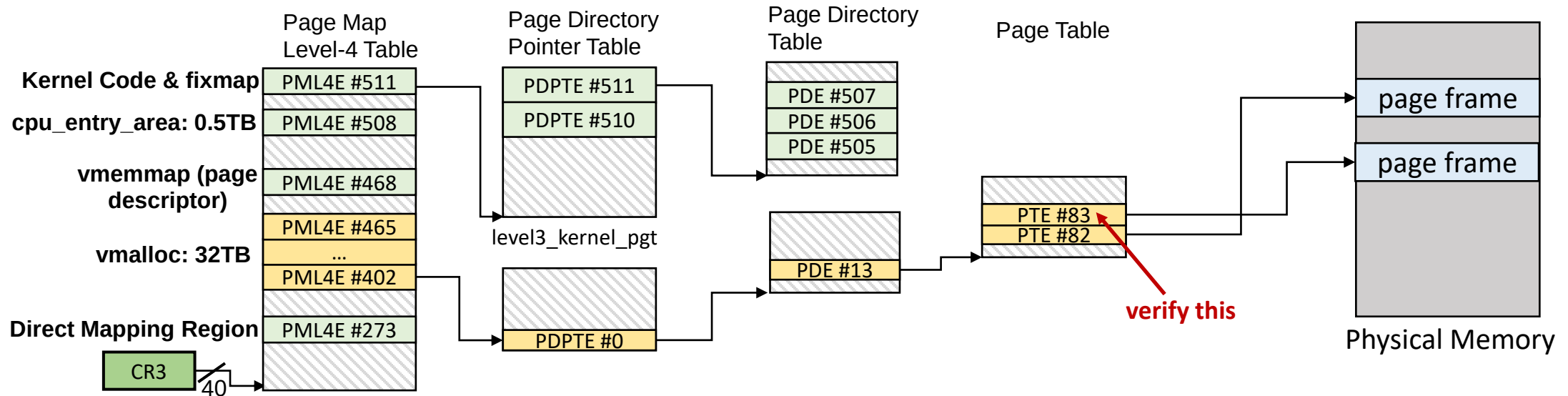
```
(gdb) x /16g area->pages
```

0xfffffc900019b9000:	0xfffffea0004090680	0xfffffea00040906c0
0xfffffc900019b9010:	0xfffffea0004090700	0xfffffea0004090740
0xfffffc900019b9020:	0xfffffea0004090780	0xfffffea00040907c0
0xfffffc900019b9030:	0xfffffea0004090000	0xfffffea0004090040
0xfffffc900019b9040:	0xfffffea0004090080	0xfffffea00040900c0
0xfffffc900019b9050:	0xfffffea0004090100	0xfffffea0004090140
0xfffffc900019b9060:	0xfffffea0004090180	0xfffffea00040901c0
0xfffffc900019b9070:	0xfffffea0004090200	0xfffffea0004090240

PTE #82 Verification

- Virtual address of the page descriptor: `0xfffffea00040907c0`
- PFN (Page Frame Number): $(0xfffffea00040907c0 - 0xfffffea00000000000) / 64 = 0x10241f$
- Page physical address: $\text{PFN} \ll 12 = 0x10241f \ll 12 = \text{0x10241f000}$

Example: vmalloc size = 8MB: Page Table Configuration



```
(gdb) bt 1
#0  __vmalloc_area_node (node=-1, prot=..., gfp_mask=11458,
    area=0xffff8881021dc580)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:253
0
(More stack frames follow...)
(gdb) p /x *((pgd_t *) 0xffff8881021b7000 + 82)
$17 = {
    pgd = 0x800000010241f063
}
(gdb) p /x *((pgd_t *) 0xffff8881021b7000 + 83)
$18 = {
    pgd = 0x8000000102400063
}
```

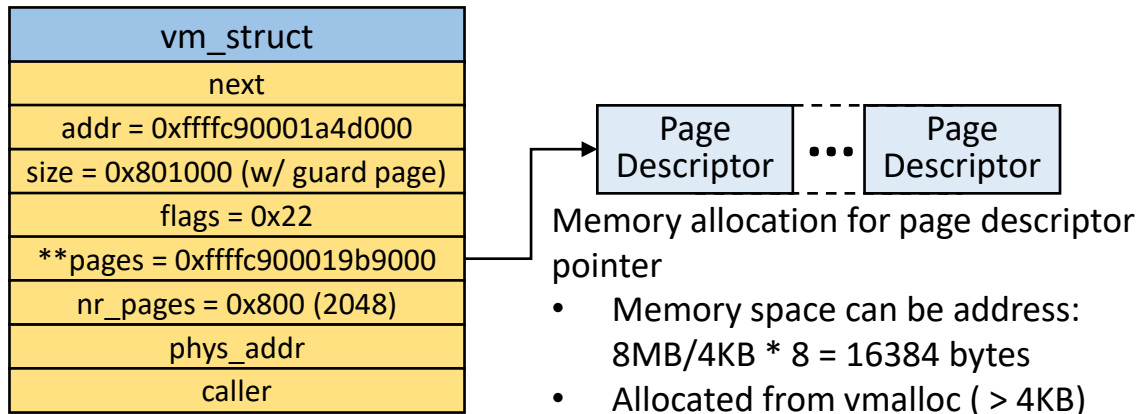
```
(gdb) x /16g area->pages
0xfffffc900019b9000: 0xfffffea0004090680 0xfffffea00040906c0
0xfffffc900019b9010: 0xfffffea0004090700 0xfffffea0004090740
0xfffffc900019b9020: 0xfffffea0004090780 0xfffffea00040907c0
0xfffffc900019b9030: 0xfffffea0004090000 0xfffffea0004090040
0xfffffc900019b9040: 0xfffffea0004090080 0xfffffea00040900c0
0xfffffc900019b9050: 0xfffffea0004090100 0xfffffea0004090140
0xfffffc900019b9060: 0xfffffea0004090180 0xfffffea00040901c0
0xfffffc900019b9070: 0xfffffea0004090200 0xfffffea0004090240
```

PTE #83 Verification

- Virtual address of the page descriptor: 0xfffffea0004090000
- PFN (Page Frame Number): $(0xfffffea0004090000 - 0xfffffea00000000000) / 64 = 0x102400$
- Page physical address: $PFN \ll 12 = 0x102400 \ll 12 = 0x102400000$

vmalloc users/scenario

- Array size > PAGE_SIZE (4KB)
 - arr[0], arr[1]....arr[n] → Need contiguous memory for array indexing
 - Example: 8MB memory allocation (for page descriptor) from vmalloc
 - Page descriptor list (vm_struct->pages) requires contiguous memory for array indexing



```
static void *__vmalloc_area_node(struct vm_struct *area, gfp_t gfp_mask,
                                pgprot_t prot, int node)
{
    const gfp_t nested_gfp = (gfp_mask & GFP_RECLAIM_MASK) | __GFP_ZERO;
    unsigned int nr_pages = get_vm_area_size(area) >> PAGE_SHIFT;
    unsigned long array_size;
    unsigned int i;
    struct page **pages;

    array_size = (unsigned long)nr_pages * sizeof(struct page *);
    gfp_mask |= __GFP_NOWARN;
    if (!(gfp_mask & (GFP_DMA | GFP_DMA32)))
        gfp_mask |= __GFP_HIGHMEM;

    /* Please note that the recursion is strictly bounded. */
    if (array_size > PAGE_SIZE) {
        pages = __vmalloc_node(array_size, 1, nested_gfp, node,
                                area->caller);
    } else {
        pages = kmalloc_node(array_size, nested_gfp, node);
    }

    if (!pages) {
        free_vm_area(area);
        return NULL;
    }

    area->pages = pages;
    area->nr_pages = nr_pages;
}

mm/vmalloc.c 2472,1
```

vmalloc users/scenario

- Virtually-mapped stack (VMAP_STACK=y)
 - Use virtually-mapped stack with guard page: kernel stack overflow can be detected immediately.

/ # ls → clone() system call

```
(gdb) bt
#0  __vmalloc_node_range (size=size@entry=16384, align=align@entry=16384,
    start=18446683600570023936, end=18446718784942112767,
    gfp_mask=gfp_mask@entry=3520, prot=..., vm_flags=0, node=-1,
    caller=0xffffffff810403f9 <kernel_clone+137>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:256
#1  0xffffffff8103f1d1 in alloc_thread_stack_node (node=<optimized out>,
    tsk=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/include
    /asm/cpufeature.h:175
#2  dup_task_struct (node=<optimized out>, orig=0xffff888101100000)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:86
#3  copy_process (pid=pid@entry=0x0 <fixed_percpu_data>, trace=trace@entry=0,
    node=node@entry=-1, args=args@entry=0xffffc900018a3eb8)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:19
#4  0xffffffff810403f9 in kernel_clone (args=args@entry=0xffffc900018a3eb8)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:24
#5  0xffffffff810406d0 in __do_sys_clone (clone_flags=<optimized out>,
    newsp=<optimized out>, parent_tidptr=<optimized out>,
    child_tidptr=<optimized out>, tls=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:25
#6  0xffffffff810408b0 in __se_sys_clone (tls=<optimized out>,
    child_tidptr=<optimized out>, parent_tidptr=<optimized out>,
    newsp=<optimized out>, clone_flags=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:25
#7  __x64_sys_clone (regs=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:25
```

vmalloc users/scenario

- Dynamically load kernel module: #1

```
/ # insmod vmalloc-test.ko
```

```
(gdb) bt
> PAGE_SIZE (4KB)
#0  __vmalloc_node_range (size=4136, align=1, start=18446683600570023936,
    end=18446718784942112767, gfp_mask=11456, prot=..., vm_flags=0, node=-1,
    caller=0xffffffff810a2461 <__do_sys_init_module+97>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:256
2
#1  0xffffffff810f2652 in __vmalloc_node (size=<optimized out>,
    align=align@entry=1, gfp_mask=gfp_mask@entry=11456, node=node@entry=-1,
    caller=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/include
    /asm/cpufeature.h:175
#2  0xffffffff810f29a9 in __vmalloc (size=<optimized out>,
    gfp_mask=gfp_mask@entry=11456)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:263
3
#3  0xffffffff810a2461 in copy_module_from_user (info=0xffffc900019d3e70,
    info=0xffffc900019d3e70, len=<optimized out>, umod=0x694c80)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/module.c:
    3066
#4  __do_sys_init_module (umod=0x694c80, len=<optimized out>,
    uargs=0x66ca67 "")
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/module.c:
    4103
#5  0xffffffff810a25e5 in __se_sys_init_module (uargs=<optimized out>,
    len=<optimized out>, umod=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/module.c:
    4090
#6  __x64_sys_init_module (regs=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/module.c:
    4090
#7  0xffffffff8136af32 in do_syscall_64 (nr=<optimized out>,
    regs=0xffffc900019d3f58)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/entry/c
    ommon.c:46
```

vmalloc users/scenario

- Dynamically load kernel module: #1

```
/ # insmod vmalloc-test.ko
```

```
(gdb) bt
#0  __vmalloc_node_range (size=4136, align=1, start=18446683600570023936,
    end=18446718784942112767, gfp_mask=11456, prot=..., vm_flags=0, node=-1,
    caller=0xffffffff810a2461 <__do_sys_init_module+97>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:256
#1  0xffffffff810f2652 in __vmalloc_node (size=<optimized out>,
    align=align@entry=1, gfp_mask=gfp_mask@entry=11456, node=node@entry=-1,
    caller=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/include
    /asm/cpufeature.h:175
#2  0xffffffff810f29a9 in __vmalloc (size=<optimized out>,
    gfp_mask=gfp_mask@entry=11456)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:263
#3  0xffffffff810a2461 in copy_module_from_user (info=0xffffc900019d3e70,
    info=0xffffc900019d3e70, len=<optimized out>, umod=0x694c80)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/module.c:
    3066
#4  do_svs_init_module (umod=0x694c80, len=<optimized out>,
```

```
/* Sets info->hdr and info->len. */
static int copy_module_from_user(const void __user *umod, unsigned long len,
    struct load_info *info)
{
    int err;

    info->len = len;
+-- 8 lines: if (info->len < sizeof(*(info->hdr)))-
    info->hdr = __vmalloc(info->len, GFP_KERNEL | __GFP_NOWARN);
    if (!info->hdr)
        return -ENOMEM;
```

kernel/module.c

3038,1-8

vmalloc users/scenario

- Dynamically load kernel module: #2

/ # insmod vmalloc-test.ko

```
(gdb) bt
#0  __vmalloc_node_range (size=size@entry=16384, > PAGE_SIZE (4KB)
    align=align@entry=4096,
    start=18446744072635809792, end=end@entry=18446744073692774400,
    gfp_mask=gfp_mask@entry=3264, prot=prot@entry=..., vm_flags=0, node=-1,
    caller=0xffffffff810a0c6f <layout_and_allocate+1711>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/vmalloc.c:256
#1  0xffffffff810326d6 in module_alloc (size=16384)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/kernel/
    module.c:59
#2  0xffffffff810a0c6f in move_module (info=0xffffc900019d3e70,
    info=0xffffc900019d3e70, mod=0xffffc900018ad180)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/module.c:
    3357
#3  layout_and_allocate (info=info@entry=0xffffc900019d3e70,
    flags=flags@entry=0)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/module.c:
    3550
#4  0xffffffff810a11ad in load_module (info=info@entry=0xffffc900019d3e70,
    uargs=uargs@entry=0x66ca67 "", flags=flags@entry=0)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/module.c:
    3928
#5  0xffffffff810a24d7 in __do_sys_init_module (umod=<optimized out>,
    len=<optimized out>, uargs=0x66ca67 "")
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/module.c:
    4107
#6  0xffffffff810a25e5 in __se_sys_init_module (uargs=<optimized out>,
    len=<optimized out>, umod=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/module.c:
    4090
#7  __x64_sys_init_module (regs=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/module.c:
    4090
#8  0xffffffff8136af32 in do_syscall_64 (nr=<optimized out>,
    regs=0xffffc900019d3f58)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/entry/c
    ommon.c:46
```

Reference

- Robert Love, Linux Kernel Development (3rd Edition)
- Wolfgang Mauerer, Professional Linux Kernel Architecture

backup

Some Notes

- Kernel implementation for /proc/pid/maps
 - show_map_vma()